

ADRIC PVT Lab Platform, Software Manual

ADRIC / PVT Engineering

v0.2.0-flash-recomb-1-ge2202a7

Contents

Abstract	3
Chapter 1: Introduction	4
1.1 What This Platform Is	4
1.2 Design Philosophy	4
1.3 Architecture Overview	5
1.4 Current Scope	6
Chapter 2: Installation and Running the Platform	8
2.1 Requirements	8
2.2 Installation	8
2.3 Running the Web Application	8
2.4 The Command-Line Interface	9
2.5 Running the Tests	10
2.6 Repo Layout Reference	11
Chapter 3: Core Concepts	13
3.1 The Dual Pressure Basis	13
3.2 Units Module Conventions	13
3.3 The Katz-Firoozabadi Component Library	14
3.4 CompositionStream	15
3.5 Sample, Study, and CrossRef	16
3.6 The QC Engine	16
3.7 Typed Exceptions	18
Chapter 4: Correlations Reference	20
4.1 Pseudocritical Properties (pvt/correlations/pseudocritical/)	20
4.2 Gas Compressibility Factor (Z) (pvt/correlations/zfactor/)	27
4.3 Bubble-Point Pressure (pvt/correlations/bubble_point/)	30
4.4 Gas Viscosity (pvt/correlations/viscosity/)	34
4.5 Summary Table	37
Chapter 5 - Flash Separation (Atmospheric Flash, Water-Pump Method)	38
5.1 The laboratory test	38
5.2 Data entry	39
5.3 Calculation chain	40
5.4 Mass recombination & plus fractions	42
5.5 QC applied	43
5.6 Report contents	44
Chapter 6 - Recombination / Live Oil Preparation	46
6.1 Purpose	46

6.2 Two routes, and when to use each	46
6.3 Cylinder loading	49
6.4 Actual-GOR verification	50
6.5 K-values	51
6.6 The D-018 open ruling - GOR basis direction	51
6.7 QC and report	52
Chapter 7: Excel Import	53
7.1 The Template-Import Philosophy	53
7.2 Supported Templates	53
7.3 ADRIC Flash v6.1 Cell Map	53
7.4 ADRIC LiveOil v4.1 Cell Map	55
7.5 Component Name Alias Handling	56
7.6 The C36+ Molecular Weight Override	57
7.7 Wrong-File Detection	57
7.8 Blank-Cell and Negative-Composition Error Behavior	57
7.9 What Is Deliberately Not Read	58
Chapter 8: QC Reference	59
8.1 The QC Engine Model	59
8.2 composition_normalization	60
8.3 mw_consistency	60
8.4 hoffman_crump	61
8.5 gor_actual_vs_target	62
8.6 ThresholdRegistry Defaults	63
8.7 How Pages Surface QC	63
Chapter 9: Reporting	65
9.1 The ReportRow / ReportTable Model	65
9.2 flash_tables	65
9.3 recombination_tables	66
9.4 Excel Export Styling	67
9.5 Downloads in the App	68
9.6 CLI Text Reports	68
10. Deviations Ledger and Roadmap	70
10.1 The deviations-ledger discipline	70
10.2 Current ledger: D-001 through D-018	70
10.3 Open rulings	72
10.4 Phase roadmap	72

Abstract

This manual documents the ADRIC PVT Lab Platform: an ADNOC-internal laboratory system for reservoir-fluid property calculations, built around a pure-Python calculation engine (`pvt/`) with a Streamlit front-end (`app.py, ui/`) and a parity command-line interface (`cli.py`). The lab's validated Excel workbooks are treated throughout as specifications and golden test fixtures, never as the calculator itself; every place the engine's output deliberately departs from a source workbook is recorded, with cell-level proof, in a deviations ledger.

Version covered by this manual: `v0.2.0-flash-recomb`

As of this version, the platform implements Phase 0 (repository restructure, core data model, units and constants, a 100% branch-coverage test gate on `pvt/`), Phase 1 (a 13-correlation library covering gas Z -factor, pseudo-critical properties, bubble point, and viscosity), and Phase 2 (Flash Separation and Live Oil Preparation/Recombination, end to end: engine, QC checks, Excel import of both ADRIC lab templates, a two-page Streamlit UI, ADRIC-styled report export, and CLI parity).

The chapters that follow walk through the platform's purpose and architecture, the core data model, the correlations library, the flash and recombination workflow, the QC engine, reporting, and the testing strategy that backs all of it. The closing chapter documents the deviations-ledger discipline in full — every current entry, the open rulings awaiting a decision from the PVT domain owner, and the Phase 3-5 roadmap read against what the full formula-level dissection of the ADRIC workbook set already tells us about the modules still to come.

Design reference: `docs/superpowers/specs/2026-08-09-pvt-lab-platform-design.md`.

Chapter 1: Introduction

1.1 What This Platform Is

The PVT Lab Platform (packaged as `pvt`, described in `pyproject.toml` as the “ADRIC PVT laboratory platform engine”) is a production-grade, ADNOC-internal system for a commercial PVT laboratory: calculations, quality control, analysis, and reporting across the lab’s full scope of work. The design specification names its reference class explicitly: Calsep PVTsim, Whitson+ (`docs/superpowers/specs/2026-08-09-pvt-lab-platform-design.md`, Section 1).

Two front ends sit on top of one calculation engine:

- A Streamlit web application (`app.py`, pages under `ui/pages/`), titled “ADRIC PVT Platform” in its page configuration, for interactive lab use.
- A command-line interface (`cli.py`) exposing the same engine for scripted or headless runs.

Both consume the same typed result dataclasses, so a number produced by the web app and the same input run through the CLI are computed by identical code paths: there is exactly one place addition, a correlation, or a QC threshold is implemented.

1.2 Design Philosophy

The design spec (Section 3) lays out the principles the codebase is built to, and the repository visibly follows them:

Correct physics, documented deviations. Where a source ADRIC workbook is provably wrong, the engine implements the physically correct form rather than reproducing the bug, and the deviation is recorded in `docs/excel-deviations.md` with cell-level proof: which workbook, which cell, what Excel does, what the engine does instead, and a review status (proposed until reviewed point by point with the PVT domain owner, then approved or parity-kept). As of this writing the ledger holds entries D-001 through D-011 and D-015 through D-018 (18 entries), spanning the component-library canonization, the validation posture, correlation-formula corrections (a transposed CO₂ coefficient, an inverted exponent, a missing $10^{\wedge}$ step, a hardcoded gas-constant literal), and Phase 2’s standard-condition-basis and GOR-direction choices. One entry, D-018 (the LiveOil v4.1 GOR-basis divide direction), is marked “NEEDS SWEJ RULING” rather than merely proposed: it is a case where the engine’s chosen convention could not be verified against the workbook’s own golden values, because the workbook’s shrinkage factor happened to be 1.0.

Excel workbooks as golden ground truth, never as the calculator. The lab’s validated Excel workbooks are treated as specifications and as golden test fixtures. The platform imports lab data from filled ADRIC templates by reading only the yellow input cells (never a formula-computed cell) and recomputes every derived quantity in Python; it exports reports back to Excel. Calculation authority lives entirely in Python.

Deviations-ledger discipline. Every deliberate difference from a workbook must exist in `docs/excel-deviations.md` before a deviating test is allowed to land; any other golden-parity failure is treated as a bug in the port, not a sanctioned difference. Each module’s deviations are meant to be reviewed with the PVT domain owner point by point before the behavior is considered locked (spec Section 12).

Pure engine, thin UI. `pvt/` never imports Streamlit or anything UI-adjacent; `ui/` contains presentation only, and every number the UI displays is computed inside `pvt/`. This is verifiable directly: nothing under `pvt/` imports `streamlit`, and the Streamlit AppTest smoke tests in `tests/ui/` exist specifically to keep that boundary honest.

Single responsibility, one file per concept. One correlation per module (e.g. `pvt/correlations/bubble_point/standing.py` holds only Standing's 1947 correlation), and one lab experiment per package following a `models.py / calc.py / validate.py` pattern.

Cross-test data flows as first-class objects. Today a value like `Psat` gets hand-retyped from one workbook into the next. The platform's `Study` dataclass is designed to hold `CrossRef` references instead, so a downstream test consumes an upstream result explicitly and QC can flag when that reference has gone stale (spec Section 3, item 5; see Chapter 3 for the current shape of `Study/CrossRef`).

100% test coverage, enforced. `pyproject.toml` wires `--cov=pvt --cov-fail-under=100` with branch coverage into every `pytest` run; see Chapter 2 for the mechanics.

1.3 Architecture Overview

The engine/UI split is a directory split. `pvt/` is pure Python with no UI dependency; `ui/` is Streamlit-only presentation. The package map below is drawn directly from the current repository tree (not the design spec's target-state layout, which includes several packages (`adjust/`, most of the QC checks/ `catalog`, `reporting/study_report.py`) planned for later phases and not yet present).

```
pvt/                                     # pure engine, no Streamlit/UI imports
  __init__.py                           # flat re-export surface: from pvt import ...
  constants.py                          # legacy shim -> pvt.core.constants (removed after Phase 2)
  core/
    constants.py                        # canonical physical constants + conversion factors
    units.py                           # field/SI conversion functions (only place conversions live)
    components.py                       # Component, ComponentLibrary (Katz-Firoozabadi, 52 slots)
    composition.py                      # CompositionStream: mol%/wt%, normalization, MW, density
    plus_fractions.py                   # C7+/C11+/C20+/C36+ plus-fraction properties
    sample.py                           # Sample, Study, CrossRef dataclasses
    exceptions.py                       # PvtError, InputValidationError, ConvergenceError
  correlations/                          # Phase 1: empirical PVT correlations
    bubble_point/                       # standing, vasquez_beggs, glaso, almarhoun
    pseudocritical/                     # sutton, sbv, piper_mccain, wichert_aziz, erbar
    viscosity/                           # lee_gonzalez_eakin, jossi_stiel_thodos, critical_volumes
    zfactor/                             # dak (Dranchuk-Abou-Kassem), hall_yarborough
  experiments/                           # one package per lab test
    flash/                              # atmospheric flash separation (models/calc/validate/recombine)
    recombination/                      # separator recombination: volumetric (SF/FF) + molar routes
      compressibility.py                 # effective_c_o: oil compressibility, constant/polynomial
      molar.py, loading.py               # molar composition split + cylinder loading plan
  io/
    excel_import/                       # one reader per ADRIC template -> typed dataclasses
      flash_v61.py                       # ADRIC_Flash_Separation_Calc_v6.1.xlsx
      liveoil_v41.py                     # ADRIC_LiveOil_Preparation_Calc_v4.1.xlsx
  qc/
    engine.py                           # QCResult, Severity, ThresholdRegistry, grade(), worst()
    checks/                             # composition_normalization, mw_consistency, hoffman_crump
  reporting/
    tables.py                           # ReportTable/ReportRow builders (flash_tables, recombination_tables)
    excel_export.py                     # write_report: ADRIC-styled single-sheet .xlsx (or BytesIO)

ui/                                     # Streamlit presentation only, no calculation logic
  theme.py                              # v8 design tokens + inject() CSS
  common/components.py                  # page_header, metric_card, qc_pill, qc_panel, calc_steps, ...
  pages/
```

```

flash_page.py, flash_page_logic.py          # Flash Separation (SSF)
recombination_page.py, recombination_page_logic.py # Recombination / Live Oil

tests/
  unit/          # mirrors pvt/core + pvt/qc + pvt/experiments, 1:1
  golden/        # workbook-cached golden-value tests (SA-372 sample)
  fixtures/      # frozen workbook-derived fixture data + the two .xlsx templates
  ui/            # Streamlit AppTest smoke tests (outside the pvt coverage gate)

docs/
  excel-deviations.md          # the deviations ledger
  workbook-defect-review.md    # formula-level defect catalog awaiting domain-owner ruling
  superpowers/specs/          # design spec(s)
  superpowers/plans/          # phase implementation plans

app.py          # Streamlit entry point (st.navigation shell over ui/pages/)
cli.py          # command-line interface over the same engine

```

Note one deliberate deviation from the design spec’s own target layout: the spec’s Section 4 shows `app.py` and `theme.py` living under `ui/`; in the repository as built, `app.py` sits at the repository root (it is the project’s actual Streamlit entry point, referenced by `streamlit run app.py`) and `ui/theme.py` remains under `ui/`. The spec’s `pvt/core` also does not list `plus_fractions.py`, which exists in the current tree implementing the “plus-fraction properties” capability the spec’s experiments/flash description calls for; it was implemented as a shared `pvt.core` primitive (consumed via `CompositionStream`) rather than nested inside `pvt/experiments/flash/`, since plus-fraction cuts apply to any composition stream, not only a flash result. `pvt/qc/checks/` currently implements three of the many check modules the spec’s target layout lists (`composition_normalization`, `mw_consistency`, `hoffman_crump`); the rest (`buckley`, `y_function`, `material_balance`, `molar_balance`, `gor_backcalc`, `compressibility_sign`, `whitson_torp_cvd`) are reserved names for later phases. An `adjust/` package (Amyx/Carlson DL-to-flash-basis correction, endpoint re-anchoring) does not exist yet either; it belongs to Phase 3.

1.4 Current Scope

Per the design spec’s phased scope (Section 2) and confirmed against what is actually implemented in the repository:

- **Phase 0 (done).** Repository restructure; the core data model (`Component/ComponentLibrary`, `CompositionStream`, `Sample/Study/ CrossRef`); the units/constants layer; typed exceptions; test infrastructure with the 100% branch-coverage gate on `pvt/`.
- **Phase 1 (done).** The correlations layer: gas Z-factor (Dranchuk-Abou- Kassem primary, Hall-Yarborough secondary), pseudo-criticals (Sutton, SBV, Piper-McCain-Corredor, Wichert-Aziz sour correction, Erbar C7+ characterization), bubble point (Standing, Vasquez-Beggs, Glaso, Al-Marhoun), viscosity (Lee-Gonzalez-Eakin gas, Jossi-Stiel-Thodos dense gas).
- **Phase 2 (done, pending phase-wrap review).** Flash Separation and Live Oil Preparation/Recombination end to end: engine (volumetric SF/FF and molar composition-split recombination flows, an oil-compressibility model), QC checks (composition normalization, MW consistency, Hoffman-Crump crossplot), Excel import of both ADRIC templates (`flash_v61`, `liveoil_v41`), a two-page Streamlit UI in “v8” styling, `.xlsx` report export, and CLI parity (`cli.py recombine / cli.py flash`). This is the platform’s first demonstrable end-to-end slice (spec Section 2).

This is what a user of the platform today can actually do: run flash separation and recombination calculations, either by uploading a filled ADRIC workbook or by typing values into a form; get graded QC results against house thresholds; and export or print a report. Everything else described below is roadmap.

Phase roadmap (design spec, Section 2)

- **Phase 3, planned.** Constant Composition Expansion (CCE); Differential Vaporization (DV), including the Amyx/Carlson flash-basis adjustment and endpoint re-anchoring; Multi-Stage Separation (MSS); Density HPHT; Viscosity HPHT.
- **Phase 4, planned.** Constant Volume Depletion (CVD, Whitson-style material balance); MMP slim-tube analysis, including a real two-line- intersection solver the source Excel workbooks never had.
- **Phase 5, planned.** A cross-test QC Center implementing the full PVT-check catalog, and consolidated study-level reporting.

Explicitly out of scope for now, with the seams reserved rather than closed off: cubic equation-of-state (PR/SRK) flash and phase envelopes (the correlations/ + experiments/ split leaves room for a future eos/ package); multi-user server deployment (v1 runs locally via `streamlit run`, single user); the master suite's non-PVT modules (SCAL, RCA, water chemistry, geochemistry); and asphaltene/SARA/swelling studies (spec Section 2).

Chapter 2: Installation and Running the Platform

2.1 Requirements

The engine requires Python 3.12 or later. `pyproject.toml` pins this explicitly:

```
[project]
requires-python = ">=3.12"
dependencies = [
    "streamlit>=1.37.1,<2",
    "openpyxl>=3.1,<4",
]
```

Runtime dependencies are deliberately minimal: `streamlit` powers the web UI and `openpyxl` powers Excel import/export. Nothing under `pvt/` itself requires a numerical library beyond the Python standard library (`math`, `dataclasses`, `enum`).

Development dependencies, declared under the `dev` extra, add the test and lint toolchain:

```
[project.optional-dependencies]
dev = [
    "pytest>=8.0",
    "pytest-cov>=5.0",
    "ruff>=0.5",
    "mypy>=1.10",
]
```

2.2 Installation

Install the package in editable mode with the `dev` extra so the test and lint tools come along:

```
pip install -e ".[dev]"
```

`[tool.setuptools.packages.find]` in `pyproject.toml` restricts what `setuptools` discovers to `pvt*` and `ui*`; `app.py` and `cli.py` are top-level scripts, not part of an installed package, and are simply run from a checkout of the repository.

2.3 Running the Web Application

Launch the Streamlit app from the repository root:

```
streamlit run app.py
```

`app.py` also supports being launched directly, in which case it re-execs itself through Streamlit:

```
python app.py
```

app.py is a thin `st.navigation` shell: it calls `st.set_page_config`, injects the theme via `ui.theme.inject()`, and registers two pages, each implemented as its own module under `ui/pages/`:

Page title (navigation)	Module	What it does
Flash Separation (SSF)	<code>ui/pages/flash_page.py</code>	Two entry modes: upload a filled <code>ADRIC_Flash_Separation_Calc_v6.1.xlsx</code> workbook, or fill a manual-entry form mirroring <code>FlashVolumetrics</code> 's fields plus an optional composition editor seeded with the 52 Katz-Firoozabadi component codes; both then render metric cards, QC pills, the Hoffmann-Crump crossplot, a calculation-steps expander, and a report download.
Recombination / Live Oil	<code>ui/pages/recombination_page.py</code>	Two tabs: Volumetric (SF/FF) , a form-driven Carlsen & Whitson multi-stage recombination flow with oil-charging pressure/compressibility; and Molar (composition) , manual GOR/basis/shrinkage/density/MW inputs or an uploaded <code>ADRIC_LiveOil_Preparation_Calc_v4.1.xlsx</code> workbook, driving the molar composition split and cylinder loading plan.

A local theme file, `.streamlit/config.toml`, sets the “v8” palette (primary color `#0047BB`) that `ui/theme.py` builds on. Both filled `ADRIC` template workbooks used for manual testing/demo purposes are checked into `tests/fixtures/workbooks/`.

2.4 The Command-Line Interface

`cli.py` exposes the same engine as the web app through two subcommands, `recombine` and `flash` (`cli.py`'s `build_parser()`):

```
python cli.py --help
python cli.py recombine --help
python cli.py flash --help
```

recombine: multi-stage separator recombination

Takes typed inputs directly on the command line (no workbook needed). The required Stage 1 arguments are `--gor`, `--p_sep`, `--t_sep`, `--z_sep`, plus the recombination-side `--v_live`, `--p_recomb`, `--t_recomb` (`--z_recomb` defaults to 1.0, `--sf` defaults to 1.00, `--stages` defaults to 1 and accepts 1, 2, or 3). Stage 2/3 conditions (`--gor2/--p2/--t2/--z2`, `--gor3/--p3/--t3/--z3`) become required only when `--stages` selects them. An optional Standing bubble-point estimate activates when both `--api` and `--sg_gas` are supplied (`--t_res` overrides the temperature used, defaulting to the Stage 1 temperature). `--units` selects field (`psia/°F/scf/STB`, the default) or `si` (`bara/°C/sm3/sm3`).

Single stage (field units)

```
python cli.py recombine --gor 850 --p_sep 815 --t_sep 145 --z_sep 0.855 \
```

```
--v_live 300 --p_recomb 5014.7 --t_recomb 200 --z_recomb 0.820
```

```
# Two-stage with a Standing Pb estimate
```

```
python cli.py recombine --gor 850 --p_sep 800 --t_sep 140 --z_sep 0.865 \  
    --stages 2 --gor2 50 --p2 65 --t2 100 --z2 0.977 \  
    --v_live 300 --p_recomb 5014.7 --t_recomb 200 --z_recomb 0.820 \  
    --api 42 --sg_gas 0.72
```

```
# SI units
```

```
python cli.py recombine --units si --gor 151.4 --p_sep 55.8 --t_sep 62.8 --z_sep 0.865 \  
    --v_live 300 --p_recomb 346.7 --t_recomb 93.3 --z_recomb 0.820
```

recombine validates inputs with `validate_multistage` and prints one error per line to stderr (exit code 1) if validation fails, rather than running a calculation on bad input.

flash: import a filled ADRIC Flash v6.1 workbook

Takes exactly one argument, `--workbook`, and runs the full import → calculate → mass-basis recombine → QC → report chain:

```
python cli.py flash --workbook path/to/ADRIC_Flash_Separation_Calc_v6.1.xlsx
```

flash imports through `pvt.io.excel_import.flash_v61.read`, runs `pvt.experiments.flash.calc.calculate`, recombines the oil/gas streams by mass (`pvt.experiments.flash.recombine.recombine_mass`), grades composition normalization and MW consistency QC checks, and prints the resulting report tables as fixed-width text via `pvt.reporting.tables.flash_tables`. A malformed workbook or unreadable file path is reported with error: lines on stderr and exit code 1, rather than a Python traceback.

Both subcommands share the same fixed-width report formatting helpers (`_format_report_tables`, `_row`, `_section`, `_rule` in `cli.py`).

2.5 Running the Tests

pytest

pyproject.toml's `[tool.pytest.ini_options]` fixes `testpaths = ["tests"]` and sets:

```
addopts = "--cov=pvt --cov-report=term-missing --cov-fail-under=100"
```

```
[tool.coverage.run]
```

```
branch = true
```

This is the platform's central quality gate: pytest fails the build if branch coverage of the `pvt/` package drops below 100%. Coverage is scoped to `pvt/` only; `cli.py` and everything under `ui/` sit outside the `--cov=pvt` gate, but both are still exercised by their own dedicated test suites (`tests/unit/test_cli.py`; the Streamlit AppTest smoke tests under `tests/ui/`).

The suite is organized in three tiers, matching the design spec's testing strategy (Section 8):

1. **Unit-sanity** (`tests/unit/`), mirroring `pvt/core`, `pvt/qc`, `pvt/correlations`, and `pvt/experiments` 1:1: analytic limits and physical-behavior checks (unit round-trips are identity, degenerate inputs raise `InputValidationError`, threshold-grading boundaries land correctly, and so on).
2. **Golden** (`tests/golden/`), keyed to a single reference sample (SA-372; see `tests/fixtures/sa372.py` / `sa372_flash.py`): exact values cached from the source ADRIC workbooks, run across the full flash and recombination chains, plus the Excel importers themselves.
3. **Deviation**, one test per `docs/excel-deviations.md` entry, asserting the engine's correct value at the exact point it deliberately departs from a source workbook.

CI and the lint/type-check gate

`.github/workflows/ci.yml` runs on every push to `main/feature/**` and on every pull request into `main`, on `ubuntu-latest` with Python 3.12:

```
- run: pip install -e ".[dev]"
- run: ruff check pvt tests ui app.py cli.py
- run: mypy pvt
- run: pytest
- run: python -c "import pvt, ui, cli"
```

There is no `pytest -W error` flag or `filterwarnings` configuration in this repository (none was found in `pyproject.toml` or elsewhere); the platform's actual warning discipline is enforced at the linter/type-checker layer instead: `ruff check` and `mypy pvt` are both required, zero-issue CI steps (`[tool.mypy]` additionally sets `disallow_untyped_defs = true` and `warn_unused_ignores = true`, so an unnecessary `# type: ignore` is itself a failure). Runtime warnings.`warn(...)` calls that do exist in the engine (range-guard warnings in `standing.py`, `vasquez_beggs.py`, and `erbar.py`, for out-of-correlation-range inputs) are deliberate, documented behavior with dedicated tests that assert on the warning text, not something the test suite silences or is meant to be clean of. The final `import-smoke` step (`python -c "import pvt, ui, cli"`) guards the pure-engine/thin-UI boundary directly: it fails if `ui` cannot be imported (e.g. a missing Streamlit dependency) as much as if `pvt` cannot.

Before committing, run the same sequence locally:

```
python3 -m pytest                                # full suite + 100% pvt coverage gate
python3 -m ruff check pvt tests ui app.py cli.py
python3 -m mypy pvt
python3 -c "import pvt, ui, cli"                  # import-smoke, matches the CI step
```

2.6 Repo Layout Reference

Path	Contents
<code>pvt/</code>	Pure calculation engine. No Streamlit or UI import anywhere in this tree.
<code>pvt/core/</code>	Constants, unit conversions, <code>Component/ComponentLibrary</code> , <code>CompositionStream</code> , plus-fraction properties, <code>Sample/Study/CrossRef</code> , typed exceptions.
<code>pvt/correlations/</code>	Phase 1 empirical correlations: <code>bubble_point/</code> , <code>pseudocritical/</code> , <code>viscosity/</code> , <code>zfactor/</code> .
<code>pvt/experiments/</code>	One package per lab test (<code>flash/</code> , <code>recombination/</code>), each following a <code>models.py</code> / <code>calc.py</code> / <code>validate.py</code> pattern.
<code>pvt/io/excel_import/</code>	<code>flash_v61.py</code> , <code>liveoil_v41.py</code> : one reader per ADRIC template.
<code>pvt/qc/</code>	<code>engine.py</code> (severity grading, threshold registry) and <code>checks/</code> (individual QC check modules).
<code>pvt/reporting/</code>	<code>tables.py</code> (schema-driven report tables) and <code>excel_export.py</code> (ADRIC-styled <code>.xlsx</code> writer).
<code>ui/</code>	Streamlit presentation layer: <code>theme.py</code> , <code>common/components.py</code> , <code>pages/</code> .
<code>tests/unit/</code>	Mirrors <code>pvt/core</code> + <code>pvt/qc</code> + <code>pvt/experiments</code> + <code>pvt/correlations</code> , 1:1.
<code>tests/golden/</code>	Workbook-cached golden-value tests keyed to the SA-372 reference sample.

Path	Contents
tests/fixtures/	Frozen fixture data (sa372.py, sa372_flash.py) and the two filled .xlsx templates under workbooks/.
tests/ui/	Streamlit AppTest smoke tests, outside the pvt coverage gate.
docs/excel-deviations.md	The deviations ledger.
docs/workbook-defect-review.md	Formula-level workbook defect catalog awaiting a point-by-point ruling.
docs/superpowers/specs/	Design specification(s).
docs/superpowers/plans/	Phase implementation plans.
app.py	Streamlit entry point (root of the repository, not under ui/).
cli.py	Command-line interface (recombine, flash subcommands).
pyproject.toml	Packaging, pytest/coverage, ruff, and mypy configuration.
.github/workflows/ci.yml	CI pipeline: ruff check, mypy, pytest, import-smoke.

Chapter 3: Core Concepts

This chapter documents the primitives every calculation module in `pvt/` is built from: constants, units, the component library, composition arithmetic, the sample/study provenance model, the QC engine, and the engine’s typed exceptions. Everything here lives under `pvt/core/` and `pvt/qc/engine.py`.

3.1 The Dual Pressure Basis

`pvt/core/constants.py` is, in its own words, the “single source of truth for all PVT calculations”, and every module imports constants from here rather than embedding a numeric literal. Its module header calls out one design decision explicitly enough to quote in full, because getting it wrong silently shifts results:

Dual-basis design — this module deliberately carries TWO standard-condition pressures, and each constant is pinned to whichever one its source workbook uses: - 14.73 psia — the volumetric standard (`P_STD_PSIA`). Used for gasometer / GOR conversions and the ADRIC lab sheets (`CC_PER_STB`, `CC_PER_SCF`, etc.). - 14.696 psia — the atmosphere & gas-constant basis (`P_ATM_PSIA`, 1 atm). Used for `psig`→`psia` conversions, `R` values, and `SCF_PER_LBMOL`. Do not “simplify” by collapsing these to one value — mixing the two bases silently shifts results by ~0.2%.

Concretely:

$$P_{\text{STD}} = 14.73 \text{ psia} \quad P_{\text{ATM}} = 14.696 \text{ psia} \quad (1 \text{ atm})$$

`P_STD_PSIA` (14.73 psia) is the ADRIC lab’s volumetric standard: it is the basis for `CC_PER_SCF` (28,316.85), `CC_PER_STB` (158,987.29, “NIST: 158987.294928 cc; lab sheets canonize to 158987.29”), and `SCF_STB_TO_CC_CC`. `P_ATM_PSIA` (14.696 psia, exactly one physical atmosphere) is the basis `units.psig_to_psia/units.psia_to_psig` default to, and it is what `SCF_PER_LBMOL` (379.482) is derived from. The module’s docstring for `SCF_PER_LBMOL` makes the risk of confusing the two concrete: computing the molar volume as $R \cdot T_{\text{STD,R}} / P_{\text{ATM}} = 10.7316 \times 519.67 / 14.696 \approx 379.484$ (rounded to the canonized 379.482) is correct; computing the same quantity against `P_STD_PSIA` instead gives ≈ 378.61 scf/lbmol: “a different, also-valid number” for a different physical basis, not a rounding error. `docs/excel-deviations.md` D-015 documents that the pre-restructure app used the wrong pairing (`P_std=14.696` psia with `CC_PER_STB=158987.1`), shifting recombination outputs by roughly 0.2% against the canonical lab basis now in force.

The temperature standard is $T_{\text{STD}} = 60^\circ\text{F} = 519.67^\circ\text{R} = 288.7056 \text{ K}$ (`T_STD_F`, `T_STD_R`, `T_STD_K`).

One correlation keeps its own, third, reference pressure entirely outside this module: the Hoffman-Crump QC check (Section 3.6) uses 14.7 psia, “the older, rounder reference pressure the original correlation was published with”, deliberately not `P_STD_PSIA` and not `P_ATM_PSIA`; it is kept module-local in `pvt/qc/checks/hoffman_crump.py` rather than added to `pvt.core.constants`, “because it belongs to this one correlation, not to the engine’s general unit system.”

3.2 Units Module Conventions

`pvt/core/units.py` states its own scope in its header: “All conversions are one-liners built strictly on `pvt.core.constants`. No numeric literals except pure math (e.g., 5/9, 32, 141.5/131.5).” Every conversion

function is a single expression built from a named constant, never a re-derived or hardcoded factor.

Temperature. Fahrenheit, Rankine, Celsius, and Kelvin are all represented, with direct conversions where the module provides them:

$$^{\circ}R = ^{\circ}F + 459.67 \quad K = ^{\circ}C + 273.15$$

f_to_r, r_to_f, f_to_c, c_to_f, c_to_k, k_to_c, and a direct f_to_k (= (temp_f + RANKINE_OFFSET) * 5 / 9) are provided; there is no direct r_to_k/k_to_r pair; a Rankine-Kelvin conversion currently has to go through Fahrenheit or Celsius.

Pressure. psig_to_psia/psia_to_psig default their atmospheric offset to P_ATM_Psia (14.696 psia, Section 3.1) but accept an explicit override; bara_to_psia/psia_to_bar use PSIA_PER_BAR (14.5038); mbar_to_psia uses the lab's own mbar/psia pair, P_STD_Psia / P_STD_MBAR (14.73 / 1015.5981).

Volume. scf_to_cc/cc_to_scf, stb_to_cc/cc_to_stb, and the ratio converters scf_stb_to_cc_cc/cc_cc_to_scf_stb all route through CC_PER_SCF, CC_PER_STB, and SCF_STB_TO_CC_CC respectively, the 14.73 psia lab basis throughout, per Section 3.1.

Density and API gravity. This is the one place the module's docstring calls out a deliberate, named house convention rather than a strict physical conversion. api_from_density_g_cc:

```
def api_from_density_g_cc(rho: float) -> float:
    """
    API gravity from density.

    House convention: treats g/cc at 60 °F as SG 60/60 (all ADRIC sheets do);
    `sg_from_density_g_cc` gives the strict conversion.
    """
    return (141.5 / rho) - 131.5
```

$$API = \frac{141.5}{\rho} - 131.5$$

Strictly, API gravity is defined from specific gravity (a dimensionless ratio to water density), not from density in g/cc directly; the ADRIC workbooks numerically substitute the g/cc value at 60 °F for that ratio (since water is close to, but not exactly, 1.000 g/cc at 60 °F; WATER_DENSITY_60F_G_CC is 0.9991). density_g_cc_from_api inverts the same house formula ($\rho = 141.5 / (API + 131.5)$), and sg_from_density_g_cc ($SG = \rho / 0.9991$) is provided as the strict alternative when a caller needs the distinction rather than the house shortcut.

3.3 The Katz-Firoozabadi Component Library

pvt/core/components.py defines a frozen Component dataclass (code, name, mw, liquid_density_g_cc, tb_r, pc_psia, tc_r, plus a derived molar_volume_cc property = mw / liquid_density_g_cc) and a ComponentLibrary collection built from a fixed table of rows.

The canonical instance, KATZ_FIROOZABADI, holds exactly 52 components: from H2 and light gases (H2S, CO2, N2, C1...nC4) through the aromatics/cyclics (MCP, Benzene, CycloC6, MCH, Toluene, EBenzene, the xylenes, TMB124) to the single-carbon-number cuts C7 through C35 and a lumped C36+. The module's comment block states its provenance directly: "Data source: ADRIC_Flash_Separation_Calc_v6.1.xlsx, Component_Properties sheet."

The SCN rows use "the workbook's rounded n-alkane MW variant (e.g. C7 100.204, C11 156.0) rather than Katz-Firoozabadi's generalized-fraction MWs, the workbook is the source of record for this table per D-001." That cross-reference is docs/excel-deviations.md D-001: two ADRIC workbooks (LiveOil v4.1 and Flash v6.1) carry slightly different Component_Properties values for the same components (C7 100.205 vs 100.204; H2S 34.082 vs 34.0809; C36+ default 635 vs 636.4). The engine canonizes on one table, the Flash v6.1 values, rather than carrying

two parallel libraries; per-study variation is handled the one way the lab actually varies it, not by picking a different base table.

`ComponentLibrary.codes` returns component codes as an immutable tuple (a copy, not the internal storage) specifically “so callers cannot mutate the library’s singleton state by mutating what they get back from this property”; `KATZ_FIR00ZABADI` is a shared module-level instance consumed by every experiment, so this guard matters.

The C36+ override. `ComponentLibrary.with_c36_mw(mw)` is the library’s only mutation path, and it is not a mutation at all: it returns a *new* `ComponentLibrary` with an isolated, `dataclasses.replaced` C36+ `Component`, while every other component dict entry is shared by reference with the original. This is deliberately “the only editable property, matching lab convention” (design spec Section 5): a study’s plus-fraction molecular weight varies sample to sample and is measured/reported per sample, while the other 51 components’ properties are fixed physical constants. `pvt/io/excel_import/liveoil_v41.py` uses exactly this path, reading a per-workbook C36+ MW cell and building `KATZ_FIR00ZABADI.with_c36_mw(c36_mw)` before constructing that sample’s composition streams.

3.4 CompositionStream

`pvt/core/composition.py` defines `CompositionStream`, described in its module docstring as “the shared composition abstraction consumed by every experiment module (flash separation, recombination, CCE/CVD reports, ...)”. It is a frozen dataclass holding a library: `ComponentLibrary` and one or both of `mol_pct` / `wt_pct` mappings (component code $\rightarrow$ percentage).

Construction guards. `__post_init__` requires at least one of `mol_pct`/`wt_pct` to be present and non-empty, and requires every component code that does appear to exist in `library.codes`; both failures are collected into a single `InputValidationError` rather than raised on the first problem found.

Both bases may coexist, but only one need be supplied at construction. The docstring is explicit: “Exactly one of `mol_pct` / `wt_pct` may be supplied and non-empty at construction time, but both may be present after derivation (see `wt_from_mol()`).” `wt_from_mol()` computes a wt% basis from the normalized mol% basis (it does not mutate the frozen instance; callers build a new stream or hold the derived dict alongside).

Raw-sum guard semantics. `raw_mol_sum()` and `raw_wt_sum()` both carry an explicit caveat in their docstrings: each returns `0.0` both when its basis is present but genuinely sums to zero, *and* when that basis was never supplied at all; the method cannot tell the two apart. Callers that need to distinguish “absent basis” from “a real zero-sum composition” (to raise an accurate diagnostic) go through `_require_mol_basis()` / `_require_wt_basis()` instead, which raise a precise `InputValidationError` (“no mol% basis provided” / “no wt% basis provided”) when the basis mapping itself is missing or empty, rather than falling through to a confusing “sums to zero” message.

Normalization. `normalized_mol()` / `normalized_wt()` rescale a basis so it sums to exactly 100, raising `InputValidationError` (“composition sums to zero”) if the raw sum is zero.

Molecular weight, by two independent routes. Both are used together as a cross-check (Section 3.6):

$$MW_{\text{mol}} = \frac{\sum_i z_i \cdot MW_i}{\sum_i z_i} \quad MW_{\text{wt}} = \frac{100}{\sum_i (w_i / MW_i)}$$

(`mw_from_mol()` sums against the *raw*, not normalized, mol% and divides by the raw sum, algebraically equivalent to normalizing first; `mw_from_wt()` sums against the normalized wt% basis). `mw_consistency_pct()` reports the two routes’ relative deviation:

$$\Delta MW\% = \frac{MW_{\text{mol}} - MW_{\text{wt}}}{MW_{\text{wt}}} \times 100$$

Physical properties derived from the normalized wt% basis:

$$\rho_{\text{ideal}} = \frac{\sum_i w_i}{\sum_i (w_i / \rho_i)} \quad \gamma_g = \frac{MW_{\text{mol}}}{MW_{\text{air}}}$$

`liquid_density_ideal_g_cc()` is ideal-mixing liquid density (no excess- volume correction); `gas_gravity()` divides the mol-route mixture MW by AIR_MW (28.964 g/mol).

A related module, `pvt/core/plus_fractions.py`, computes aggregate properties (mol%, wt%, MW, density) for a plus-fraction cut (C7+, C11+, C20+, C36+) of a `CompositionStream`. Cut boundaries are positional on the library’s fixed 52-slot order, not name-pattern matching: “a cut is every component at or after its start code,” which is what lets C7+ exclude the cyclics that sort before C7 in the table (MCP, Benzene, CycloC6) while including the ones that sort after it (MCH, Toluene), matching the Flash workbook’s `Plus_Properties_Report` convention.

3.5 Sample, Study, and CrossRef

`pvt/core/sample.py` defines three dataclasses. Its module docstring states the intent directly: “CrossRef bridges upstream test results consumed downstream with provenance, replacing manual retyping between workbooks. Units are carried in source_field names.”

`Sample` (a plain, mutable dataclass, not frozen) carries lab/well identity: `sample_id`, `well`, `field_name`, `reservoir`, `depth_ft_md`: `float | None`, `fluid_type`, `cylinder`, plus optional `client` and `project`. This is the one dataclass from this module in active use across the current codebase today: every Excel importer, the CLI, `pvt/reporting/excel_export.py`, and the UI’s shared components all construct or consume a `Sample`.

`Study` bundles a `Sample` with reservoir conditions (`reservoir_p_psig`, `reservoir_t_f`) and four optional `CrossRef` fields: `psat`, `density_at_psat`, `rs_flash`, `bo_flash`. `CrossRef` itself is (`value`, `source_test`, `source_field`, `note=""`): a number plus a record of which upstream test and field it came from, and an optional free-text note.

As of Phase 2, `Study` and `CrossRef` exist as data-model primitives and are covered by their own unit tests (`tests/unit/test_sample.py`), but no experiment or importer currently constructs or consumes one; the Flash v6.1 importer’s own module docstring notes this directly, describing reservoir P/T and saturation-pressure cells on the workbook that are read visually by a technician but “have no home until `Study/CrossRef` are wired in a later phase.” Wiring `Study/CrossRef` into real cross-test data flow (e.g. CCE’s Psat feeding DV/MSS without manual retyping, design spec Section 3 item 5) is future work beyond the current Flash/Recombination slice.

3.6 The QC Engine

`pvt/qc/engine.py` is, per its own module docstring, the “shared vocabulary for every check module under `pvt.qc.checks`”: grade a numeric deviation against a two-band threshold, carry the result in a typed `QCResult`, and roll many results up to a worst-case severity for a study-level verdict.

Severity is a three-level `StrEnum`, ordered best to worst: `PASS < REVIEW < FAIL`.

Grading. `grade(value, review_at, fail_at, *, absolute=True)` compares the (optionally absolute-valued) input against the two band edges:

$$\text{severity} = \begin{cases} \text{PASS} & |v| \leq \text{review_at} \\ \text{REVIEW} & \text{review_at} < |v| \leq \text{fail_at} \\ \text{FAIL} & |v| > \text{fail_at} \end{cases}$$

Band edges are inclusive downward: a value landing exactly on `fail_at` grades `REVIEW`, not `FAIL`. `worst(results)` reduces any iterable of `QCResult` to the single worst `Severity` present, defaulting to `PASS` on an empty iterable.

QCResult is a frozen dataclass: `check_id`, `severity`, `value`: `float | None`, `threshold` (a human-readable band description string), `message`.

ThresholdRegistry holds one (review_at, fail_at) pair per check_id, seeded from DEFAULTS and independently overridable per study via override(check_id, review_at, fail_at, note); every override is appended to a .audit log carrying the caller's note, so a report can explain why a given study's bands differ from the house defaults. The class docstring is explicit that these defaults "come from the ADRIC house conventions," with one named exception.

The current DEFAULTS table, reproduced from pvt/qc/engine.py:

check_id	review_at	fail_at	Source	Consumed by (Phase 0-2)
composition_sum	0.5	2.0	ADRIC house convention (design spec Sections 5, 9: "±0.5 PASS / ±2 REVIEW")	pvt/qc/checks/composition_normal
mass_balance_pct	2.0	3.0	ADRIC house convention (design spec Section 9: "mass balance 2/3%")	not yet consumed by an implemented check module
molar_balance_pct	2.0	3.0	ADRIC house convention (design spec Section 9: "molar balance 2/3%")	not yet consumed by an implemented check module
z_deviation_pct	2.0	5.0	ADRIC house convention (design spec Section 9: "Z deviation 2/5%")	not yet consumed by an implemented check module
density_rsd_pct	0.5	1.0	ADRIC house convention (design spec Section 9: "density %RSD 0.5/1")	not yet consumed by an implemented check module
viscosity_vs_sim_pct	2.0	5.0	ADRIC house convention (design spec Section 9: "viscosity vs sim 2/5%")	not yet consumed by an implemented check module
mmp_mass_balance_pct	5.0	5.0	ADRIC house convention (design spec Section 9: "MMP mass balance ±5%", a single band, represented here as equal review/fail edges)	not yet consumed by an implemented check module
gor_actual_vs_target_pct	5.0	10.0	Documented as a house threshold in its consuming module's own docstring; not individually itemized in design spec Section 9's list	pvt/experiments/recombination/verify_actual_gor

check_id	review_at	fail_at	Source	Consumed by (Phase 0-2)
mw_consistency_pct	5.0	10.0	Documented as a house threshold in its consuming module's own docstring; not individually itemized in design spec Section 9's list	pvt/qc/checks/mw_consistency.py
hoffman_r2	0.98	0.95	Not an ADRIC house convention: the class docstring is explicit that this pair "is proposed by engineering judgment... pending Swej calibration," because the source PVT-check sheets show this crossplot visually with no numeric R^2 gate of their own	pvt/qc/checks/hoffman_crump.py

Three implementation details worth calling out precisely:

- **mmp_mass_balance_pct's (5.0, 5.0) pair is not a typo.** Because `review_at == fail_at`, `grade()` can never actually return REVIEW for this check: a value at or below 5.0% is PASS, anything above is immediately FAIL. This mirrors the spec's own single-number framing ("MMP mass balance $\pm 5\%$ ") rather than the two-band framing every other default uses.
- **hoffman_r2 is graded as an R^2 -floor, not a deviation band.** Its module (`pvt/qc/checks/hoffman_crump.py`) documents the semantics directly: `(0.98, 0.95)` reads " $R^2 \geq 0.98$ is PASS, $R^2 \geq 0.95$ is REVIEW, below that is FAIL", the opposite direction from every other threshold pair, where a *smaller* number is better. Because `grade()` itself only understands "smaller deviation is better," the check converts before calling it: it grades $1 - R^2$ against $(1 - 0.98, 1 - 0.95)$.
- **Six of the ten seeded defaults have no consuming check module yet.** `mass_balance_pct`, `molar_balance_pct`, `z_deviation_pct`, `density_rsd_pct`, `viscosity_vs_sim_pct`, and `mmp_mass_balance_pct` are all present in `ThresholdRegistry.DEFAULTS` today but are not read by any check module in `pvt/qc/checks/` as of Phase 2; they are reserved for check modules the design spec's target `qc/checks/` layout names (`material_balance.py`, `molar_balance.py`, a Z-factor/DAK-vs-Hall- Yarborough deviation check, etc.) that belong to later phases.

3.7 Typed Exceptions

`pvt/core/exceptions.py` defines the engine's exception hierarchy, all deriving from one base:

- **PvtError**: base class for every engine error.
- **InputValidationError(errors: list[str])**: "Raised when calc inputs fail validation. Carries the message list." Stores the original list on `.errors` and joins it with "; " as the exception's string message, so a caller can either report every failure individually (iterating `.errors`, as `cli.py` does for both subcommands) or just print the exception.
- **ConvergenceError(message, *, iterations: int, residual: float)**: raised when an iterative solver (Newton iteration in the DAK Z-factor correlation, Hall-Yarborough's reduced-density solve) fails to converge.

Both diagnostic values are stored as attributes and folded into the formatted message: `f"{message} (iterations={iterations}, residual={residual:.3e})"`.

Every validation failure in the engine (a malformed composition, a degenerate least-squares fit in the Hoffman-Crump check, a blank or negative cell caught by an Excel importer) surfaces as one of these two typed exceptions rather than a bare `ValueError`, `KeyError`, or `ZeroDivisionError`, which is what lets `cli.py` and the Streamlit pages catch `InputValidationError` specifically and render its `.errors` list as user-facing messages instead of a traceback.

Chapter 4: Correlations Reference

This chapter documents every empirical correlation implemented under `pvt/correlations/`: pseudocritical properties, gas compressibility factor (Z), bubble-point pressure, and gas viscosity. Each section follows the same structure: Purpose, Equation(s), Inputs & units, Validity & guards, Source, Function signature, and Anchoring (how the implementation is pinned against a published value, a source spreadsheet cell, or a hand/VBA trace).

Every equation below was transcribed directly from the corresponding module in `pvt/correlations/` and checked term-by-term against the source code (not retyped from memory of the textbook form), so engineers can diff it against their own references with confidence. Where the engine deliberately diverges from a source workbook or VBA macro, the deviation is called out and cross-referenced to its ledger entry in `docs/excel-deviations.md` (the “D-0xx” identifiers).

4.1 Pseudocritical Properties (`pvt/correlations/pseudocritical/`)

4.1.1 Sutton (1985)

Purpose. Estimate sweet-gas pseudocritical temperature and pressure from gas specific gravity alone (no composition required).

Equation(s).

$$p_{pc} = 756.8 - 131.0 \gamma_g - 3.6 \gamma_g^2$$

$$T_{pc} = 169.2 + 349.5 \gamma_g - 74.0 \gamma_g^2$$

Inputs & units.

Symbol	Meaning	Units
γ_g	Gas specific gravity (air = 1)	dimensionless

Returns (T_{pc}, p_{pc}) in (deg R, psia).

Validity & guards. Raises `InputValidationError` unless $0.55 \leq \gamma_g \leq 2.0$. This is a sweet-gas correlation (no H₂S/CO₂ term); the module docstring directs callers to apply `wichert_aziz.correct` to its output for sour gas.

Source. Sutton, R.P., SPE 14265 (1985). Coefficients as used in the ADRIC CVD workbook (Additional_QC!E9/F9) and DV v5 Additional_QC.

Function signature.

```
def pseudo_criticals(gas_gravity: float) -> tuple[float, float]:
    """Return (Tpc [R], Ppc [psia])."""
```

Anchoring. tests/unit/correlations/test_sutton.py::test_known_value_gamma_07 recomputes both formulas independently at $\gamma_g = 0.7$ and asserts exact agreement (rel=1e-12); this is a formula self-check, not an external published or workbook value. test_physical_trend checks the expected monotonic direction (heavier gas raises T_{pc} , lowers p_{pc}), and test_out_of_range_rejected parametrizes over $\gamma_g \in \{0.0, 0.4, 2.5, -1.0\}$ to confirm the range guard.

4.1.2 Stewart-Burkhardt-Voo (1959)

Purpose. Compute mixture pseudocriticals from a full mole composition using the SBV J/K mixing rules.

Equation(s). With y_i the normalized mole fraction of component i :

$$J = \frac{1}{3} \sum_i y_i \frac{T_{c,i}}{p_{c,i}} + \frac{2}{3} \left(\sum_i y_i \sqrt{\frac{T_{c,i}}{p_{c,i}}} \right)^2$$

$$K = \sum_i y_i \frac{T_{c,i}}{\sqrt{p_{c,i}}}$$

$$T_{pc} = \frac{K^2}{J}, \quad p_{pc} = \frac{T_{pc}}{J}$$

Inputs & units.

Symbol	Meaning	Units
stream	CompositionStream (mol% composition + component library)	mol%
$T_{c,i}, p_{c,i}$	Component critical temperature / pressure (from stream.library)	deg R, psia

Returns (T_{pc}, p_{pc}) in (deg R, psia).

Validity & guards. No explicit range check in sbv.py itself; the composition is normalized via stream.normalized_mol(), and unknown component codes are rejected at CompositionStream construction time (InputValidationError), not inside this module. This is a sweet-gas correlation; apply wichert_aziz.correct to the output for sour gas.

Source. Stewart-Burkhardt-Voo (1959) pseudo-critical mixing rules, per the module docstring.

Function signature.

```
def pseudo_criticals(stream: CompositionStream) -> tuple[float, float]:
    """Return (Tpc [R], Ppc [psia]) from mole composition via SBV J/K."""
```

Anchoring. tests/unit/correlations/test_sbv.py::test_golden_equimolar_c1c2c3 is a workbook golden: "Z factor calculation.xls" cells I5/I6 for an equimolar C1/C2/C3 mix, checked at rel=2e-3 (the tolerance absorbs a small Katz-Firoozabadi-vs-workbook Tc/Pc table difference, D-001, rather than masking formula regressions). test_single_component_recovers_own_criticals is an exact identity check (rel=1e-12): for a pure C1 stream, $J = K = 1$ and (T_{pc}, p_{pc}) must recover the component's own (T_c, p_c) from the library exactly.

4.1.3 Piper, McCain & Corredor (1993), SPE 26668

Purpose. Estimate mixture pseudocriticals from either (a) gas gravity plus sour-species mole fractions, or (b) a full mole composition, using correlations that are sour-aware from the start (unlike Sutton/SBV).

Both forms solve the same J/K shape: $T_{pc} = K^2/J$, $p_{pc} = T_{pc}/J$.

Equation(s), gravity form (from_gravity).

$$J = \alpha_0 + \alpha_1 y_{H_2S} \frac{T_{c,H_2S}}{p_{c,H_2S}} + \alpha_2 y_{CO_2} \frac{T_{c,CO_2}}{p_{c,CO_2}} + \alpha_3 y_{N_2} \frac{T_{c,N_2}}{p_{c,N_2}} + \alpha_4 \gamma_g + \alpha_5 \gamma_g^2$$

$$K = \beta_0 + \beta_1 y_{H_2S} \frac{T_{c,H_2S}}{\sqrt{p_{c,H_2S}}} + \beta_2 y_{CO_2} \frac{T_{c,CO_2}}{\sqrt{p_{c,CO_2}}} + \beta_3 y_{N_2} \frac{T_{c,N_2}}{\sqrt{p_{c,N_2}}} + \beta_4 \gamma_g + \beta_5 \gamma_g^2$$

Gravity-form coefficient table ($\alpha_0 \dots \alpha_5 / \beta_0 \dots \beta_5$):

Coefficient	Value	Coefficient	Value
α_0	0.11582	β_0	3.8216
α_1	-0.4582	β_1	-0.06534
α_2	-0.90348	β_2	-0.42113
α_3	-0.66026	β_3	-0.91249
α_4	0.70729	β_4	17.438
α_5	-0.099397	β_5	-3.2191

The H2S/CO2/N2 critical properties used in these terms are pinned to the Katz-Firoozabadi (KF) library rows (H2S, CO2, N2), not a separate table copied from the paper, so that from_gravity and from_composition stay numerically consistent when fed a KF-library stream.

Equation(s), compositional form (from_composition). Let $S_J = \sum_{i \in C1..C6} y_i T_{c,i}/p_{c,i}$, $S_K = \sum_{i \in C1..C6} y_i T_{c,i}/\sqrt{p_{c,i}}$, and $F = y_{C7+} \cdot MW_{C7+}$ (the C7+ bucket's mole fraction times its molecular weight):

$$J = \alpha_0 + \alpha_1 y_{H_2S} \frac{T_{c,H_2S}}{p_{c,H_2S}} + \alpha_2 y_{CO_2} \frac{T_{c,CO_2}}{p_{c,CO_2}} + \alpha_3 y_{N_2} \frac{T_{c,N_2}}{p_{c,N_2}} + \alpha_4 S_J + \alpha_5 S_J^2 + \alpha_6 F + \alpha_7 F^2$$

$$K = \beta_0 + \beta_1 y_{H_2S} \frac{T_{c,H_2S}}{\sqrt{p_{c,H_2S}}} + \beta_2 y_{CO_2} \frac{T_{c,CO_2}}{\sqrt{p_{c,CO_2}}} + \beta_3 y_{N_2} \frac{T_{c,N_2}}{\sqrt{p_{c,N_2}}} + \beta_4 S_K + \beta_5 S_K^2 + \beta_6 F + \beta_7 F^2$$

where a sour term (e.g. $y_{H_2S} T_{c,H_2S}/p_{c,H_2S}$) evaluates to 0 if that species is absent from the composition, and

$$MW_{C7+} = \frac{\sum_{i \in C7+} y_i MW_i}{\sum_{i \in C7+} y_i}$$

unless the caller overrides it via the c7p_mw argument. The C7+ bucket is every component that is neither a sour species (H2S/CO2/N2) nor in the C1-C6 list, including naphthenes/aromatics (e.g. MCP, Benzene) by design.

Compositional-form coefficient table ($\alpha_0 \dots \alpha_7 / \beta_0 \dots \beta_7$):

Coefficient	Value	Coefficient	Value
α_0	0.052073	β_0	-0.39741
α_1	1.0160	β_1	1.0503
α_2	0.86961	β_2	0.96592

Coefficient	Value	Coefficient	Value
α_3	0.72646	β_3	0.78569
α_4	0.85101	β_4	0.98211
α_5	0.0	β_5	0.0
α_6	0.020818	β_6	0.45536
α_7	-0.0001506	β_7	-0.0037684

Index 5 is published as 0.0 (no squared-HC-sum term in this form); it is kept only for positional alignment with the gravity form's α_4/α_5 (γ_g, γ_g^2) slots.

Inputs & units.

Gravity form: γ_g (dimensionless), $y_{H_2S}, y_{CO_2}, y_{N_2}$ as mole **fractions** in $[0, 1]$ (not mole percent). Compositional form: stream (CompositionStream, mol% basis) and optional c7p_mw override (g/mol). Both return (T_{pc}, p_{pc}) in (deg R, psia).

Validity & guards. from_gravity raises InputValidationError (collecting all violations) if $\gamma_g \leq 0$, any of $y_{H_2S}, y_{CO_2}, y_{N_2} \notin [0, 1]$, or their sum exceeds 1 - explicitly guarding the mole-percent trap (e.g. passing y_co2=20 meaning 20%). from_composition has no dedicated validation function; it inherits CompositionStream's construction-time checks.

Sour-route warning (do not chain into Wichert-Aziz). Both entry points are already impurity-adjusted through their own $\alpha_2/\alpha_1/\alpha_3$ (J) and $\beta_2/\beta_1/\beta_3$ (K) sour terms. Passing their output through wichert_aziz.correct double-applies the sour correction; at 5% CO2/3% H2S this shifts T_{pc} by roughly 3.5% relative to the correct single-application result.

Source. Piper, L.D., McCain, W.D., and Corredor, J.H. (1993), SPE 26668. Coefficients above are the *published* SPE 26668 values, not the source workbook's.

Function signature.

```
def from_gravity(
    gas_gravity: float, y_h2s: float = 0.0, y_co2: float = 0.0, y_n2: float = 0.0
) -> tuple[float, float]: ...

def from_composition(
    stream: CompositionStream, c7p_mw: float | None = None
) -> tuple[float, float]: ...
```

Anchoring. test_golden_gravity_form_sweet is a workbook golden ("Z factor calculation.xls" unknown-composition sheet F4/F5, $\gamma_g = 0.737$, no impurities), rel=1e-9. test_golden_compositional_form is a workbook golden against the sour known-composition sheet I5/I6, rel=2e-3 (library Tc/Pc differ from the workbook's in the 3rd-4th significant figure). test_deviation_d003_co2_coefficient is an exact hand-computed pin (independently derived from the published J/K formula, the published $\alpha_2 = -0.90348$, and the KF library's CO2 $T_c/p_c = 547.6 \text{ R}/1071.0 \text{ psia}$, rel=1e-9; it exists specifically to catch the D-003 digit-transposition regression (workbook's $\alpha_2 = -0.09034$). test_compositional_form_h2s_present is a self-derived formula-spec pin exercising the H2S branch, rel=1e-9. test_c7_plus_bucket_includes_naphthenes_mole_weighted_mw verifies D-004 (mole-fraction-weighted C7+ MW, including non-C1-C6/non-sour species like MCP) by cross-checking the auto-computed result against an explicit c7p_mw override, and confirms the workbook's unweighted-average bug would give a materially different (wrong) answer.

4.1.4 Wichert-Aziz (1972) Sour Gas Correction

Purpose. Correct pseudocritical temperature and pressure for the presence of CO2 and H2S, given a sweet-gas (T_{pc}, p_{pc}) pair (e.g. from Sutton or SBV).

Equation(s).

$$A = y_{CO_2} + y_{H_2S}, \quad B = y_{H_2S}$$

$$\varepsilon = 120 (A^{0.9} - A^{1.6}) + 15 (B^{0.5} - B^4) \quad [^\circ R]$$

$$T'_{pc} = T_{pc} - \varepsilon$$

$$p'_{pc} = p_{pc} \frac{T'_{pc}}{T_{pc} + B(1 - B)\varepsilon}$$

Inputs & units.

Symbol	Meaning	Units
T_{pc}, p_{pc}	Uncorrected pseudocriticals	deg R, psia
y_{CO_2}, y_{H_2S}	Mole fractions	dimensionless, [0, 1]

Returns (T'_{pc}, p'_{pc}) in (deg R, psia).

Validity & guards. Raises `InputValidationError` (collecting all violations) if $y_{CO_2} \notin [0, 1]$, $y_{H_2S} \notin [0, 1]$, or $y_{CO_2} + y_{H_2S} > 1$; the guard messages explicitly call out that these are mole fractions, not mole percent.

Source. Wichert, G.C., and Aziz, K. (1972). “Calculate Z’s for Sour Gases.” *Hydrocarbon Processing*, 51(5), 119-122. As implemented in the Amoco GasProp VBA (CalculateCriticals).

Function signature.

```
def correct(tpc_r: float, ppc_psia: float, y_co2: float, y_h2s: float) -> tuple[float, float]:
```

Anchoring. `test_no_impurities_is_identity` confirms $A = B = 0$ returns the input unchanged. `test_hand_computed_case` independently recomputes ε from the closed-form expression at $A = 0.15$, $B = 0.05$ and checks both T'_{pc} and p'_{pc} against it at `rel=1e-12` (a formula self-check, not an external golden). `test_correction_lowers_both` confirms the expected physical direction.

4.1.5 Erbar C7+ Pseudocriticals (Chao-Seader lineage; Hall 1971 Vc)

Purpose. Estimate (T_c, p_c, V_c) for a C7+ pseudo-component from its molecular weight and specific gravity, for use as an input to mixture pseudocritical mixing rules (e.g. `critical_volumes.vc_mix`).

Equation(s). `c7_plus_criticals` is a line-for-line transcription of the private VBA subroutine `EstimatePseudoCriticals` (`docs/reference/gasprop_functions.bas`). Inputs are first floored: $MW < 99 \rightarrow 99$, $sg < 0.7 \rightarrow 0.74$ (each emits a `UserWarning` containing “clamped”). With $M = MW$ (post-clamp):

Boiling point (deg F), a quartic in M plus an SG-slope quartic in M :

$$bp_{base}(M) = -264.65726 + M \left(6.2374923 + M \left(-0.021451518 + M \left(4.3992405 \times 10^{-5} - 3.43845 \times 10^{-8} M \right) \right) \right)$$

$$s_x(M) = 364.9632 + M \left(-4.759161 + M \left(0.04974927 + M \left(-1.5157213 \times 10^{-4} + 1.431011 \times 10^{-7} M \right) \right) \right)$$

$$bp = bp_{base}(M) + s_x(M) (sg - 0.6)$$

If $sg > 0.86$, with $c_1 = sg - 0.86$:

$$sz = \left[(16.823557 + M(-0.071486 + 0.000998994 M)) + (65.42352 + M(0.9092107 - 0.00801609 M))c_1 \right] c_1$$

$$bp \leftarrow bp + e^{sz}$$

With bp finalized, $b_2 = bp^2$, $b_3 = bp^3$:

$$sgr_p = 0.57248636 + 0.0006948103 bp - 7.5728178 \times 10^{-7} b_2 + 3.207736 \times 10^{-10} b_3$$

$$sgr_b = 0.91610329 - 0.00025041792 bp + 3.5706705 \times 10^{-7} b_2 - 1.663182 \times 10^{-10} b_3$$

$$sgr_n = 1.9082378 - 0.0034097612 bp + 4.3083811 \times 10^{-6} b_2 - 1.85173 \times 10^{-9} b_3$$

$$x_{mp} = 45.19165 + 0.26993166 bp - 8.805269 \times 10^{-5} b_2 + 3.58456 \times 10^{-7} b_3$$

$$x_{mb} = 14.93085 + 0.407469 bp - 4.228928 \times 10^{-4} b_2 + 5.85848 \times 10^{-7} b_3$$

$$x_{mn} = 4.825517 + 0.13158172 bp + 4.2669638 \times 10^{-4} b_2 - 1.49796 \times 10^{-7} b_3$$

Paraffin/naphthene/aromatic (PNA) volume-fraction split: if $sg \leq sgr_b$,

$$vf_p = \frac{sg - sgr_b}{sgr_p - sgr_b}, \quad vf_b = 1 - vf_p, \quad vf_n = 0$$

else

$$vf_b = \frac{sg - sgr_n}{sgr_b - sgr_n}, \quad vf_n = 1 - vf_b, \quad vf_p = 0$$

Converted to mole fractions via two weighting passes:

$$q = vf_p sgr_p + vf_b sgr_b + vf_n sgr_n$$

$$wf_p = \frac{vf_p sgr_p}{q}, \quad wf_b = \frac{vf_b sgr_b}{q}, \quad wf_n = \frac{vf_n sgr_n}{q}$$

$$q' = \frac{wf_p}{x_{mp}} + \frac{wf_b}{x_{mb}} + \frac{wf_n}{x_{mn}}$$

$$x_{fp} = \frac{wf_p}{x_{mp} q'}, \quad x_{fb} = \frac{wf_b}{x_{mb} q'}, \quad x_{fn} = \frac{wf_n}{x_{mn} q'}$$

Critical temperature (deg R), a PNA-weighted blend of three cubics in bp :

$$xz_u = 727.47745 + 1.2626579 bp - 4.5330572 \times 10^{-4} b_2 + 1.23217 \times 10^{-7} b_3$$

$$xz_i = 839.54553 + 1.0776683 bp - 4.7253008 \times 10^{-4} b_2 + 2.8135443 \times 10^{-7} b_3$$

$$xz_o = 1521.9287 - 1.5416102 bp + 0.0033237804 b_2 - 1.65984 \times 10^{-6} b_3$$

$$T_c = \max(0, x_{fp} xz_u + x_{fb} xz_i + x_{fn} xz_o)$$

Critical pressure (psia), same blend structure, different cubics:

$$xz_u = 593.11935 - 1.1655109 bp + 0.001210827 b_2 - 6.92878 \times 10^{-7} b_3$$

$$xz_i = 1128.158 - 2.8264468 bp + 0.0028014571 b_2 - 9.72225 \times 10^{-7} b_3$$

$$xz_o = 2748.4398 - 9.519013 bp + 0.012696074 b_2 - 5.97439 \times 10^{-6} b_3$$

$$p_c = \max(0, x_{fp} xz_u + x_{fb} xz_i + x_{fn} xz_o)$$

Critical volume, Hall's (1971) correlation (note: uses specific gravity, not density, per the D-016 contract below):

$$V_c = 0.025 \left(\frac{M}{sg^{0.69}} \right)^{1.15}$$

Inputs & units.

Symbol	Meaning	Units
MW	C7+ molecular weight	lb/lbmol
sg	C7+ specific gravity, relative to water	dimensionless

Returns (T_c, p_c, V_c) in (deg R, psia, ft³/lbmol).

Validity & guards. Inputs are floored (not rejected): $MW < 99 \rightarrow 110$, $sg < 0.7 \rightarrow 0.74$, each emitting a `UserWarning` containing “clamped”. T_c and p_c are separately floored at 0.0 as a defensive clamp only reachable for non-physical (MW, sg) combinations far outside the correlation's intended range.

D-016: the SG-not-density contract. The VBA source (`EstimatePseudoCriticals`, line ~497) takes a C7+ *density* argument (g/cc) and converts it internally via $sg = \text{PseudoDensGmCC} / 0.999015$ before using it anywhere downstream. This module's public signature takes sg (specific gravity) directly, and uses it everywhere the VBA uses sg - i.e. the $/0.999015$ conversion is **not** applied a second time. Callers holding a raw density must convert to specific gravity themselves before calling. This is forced by the pinned characterization test ($vc == 0.025 * (mw/sg^{0.69})^{1.15}$ at $rel=1e-12$, $mw=217$, $sg=0.845$): applying the extra $/0.999015$ division would shift V_c by about 0.07%, well outside that tolerance. See `docs/excel-deviations.md` D-016.

Source. Erbar (Chao-Seader program) C7+ correlation, transcribed verbatim from the Amoco GasProp VBA (`docs/reference/gasprop_functions.bas`); critical volume via Hall, R.R. (1971).

Function signature.

```
def c7_plus_criticals(mw: float, sg: float) -> tuple[float, float, float]:
    """Return (tc_r, pc_psia, vc)."""
```

Anchoring. This correlation is proprietary Amoco/Chao-Seader lineage with no public worked example, so its primary anchor is a **VBA-trace** (transcription self-consistency), not an external published or workbook value. `test_vba_trace_mw217_sg0845` hand-traces `EstimatePseudoCriticals(PseudoMolWt=217.0, PseudoDens=GmCC=0.845)` line-by-line from the `.bas` file independently of `erbar.py` (intermediate values `bp=560.6575339349728`, `sgrp=0.7805270473601936`, `sgrb=0.8586327919042741`, `sgrn=1.024472269520219`), then asserts the Python output matches that trace to `rel=1e-12` ($T_c = 1340.2602949758316$, $p_c = 244.54023185923086$, $V_c = 13.896579108178768$). Supporting tests: `test_typical_c7plus_is_physical` (bounds sanity + exact V_c formula check), `test_monotone_in_mw`, `test_clamps_warn` (both floors fire and warn), and `test_extreme_inputs_floor_tc_and_pc_at_zero` (drives $MW = 1000$, $sg = 0.72$ to exercise the $T_c < 0/p_c < 0$ defensive floors).

4.2 Gas Compressibility Factor (Z) (pvt/correlations/zfactor/)

4.2.1 Dranchuk & Abou-Kassem (1975)

Purpose. Solve the DAK equation of state for Z by Newton iteration on the implicit residual, given pressure, temperature, and pseudocriticals.

Equation(s). With $T_{pr} = T/T_{pc}$, $p_{pr} = p/p_{pc}$:

$$C_1 = A_1 + \frac{A_2}{T_{pr}} + \frac{A_3}{T_{pr}^3} + \frac{A_4}{T_{pr}^4} + \frac{A_5}{T_{pr}^5}$$

$$C_2 = A_6 + \frac{A_7}{T_{pr}} + \frac{A_8}{T_{pr}^2}$$

$$C_3 = A_9 \left(\frac{A_7}{T_{pr}} + \frac{A_8}{T_{pr}^2} \right)$$

$$\rho_r = \frac{0.27 p_{pr}}{Z T_{pr}}$$

$$C_4 = A_{10} (1 + A_{11} \rho_r^2) \frac{\rho_r^2}{T_{pr}^3} e^{-A_{11} \rho_r^2}$$

$$F(Z) = Z - (1 + C_1 \rho_r + C_2 \rho_r^2 - C_3 \rho_r^5 + C_4) = 0$$

solved by Newton's method, $Z_{n+1} = Z_n - F(Z_n)/F'(Z_n)$, with (using $d\rho_r/dZ = -\rho_r/Z$):

$$F'(Z) = 1 + \frac{C_1 \rho_r}{Z} + \frac{2C_2 \rho_r^2}{Z} - \frac{5C_3 \rho_r^5}{Z} + \frac{2A_{10} \rho_r^2}{T_{pr}^3 Z} e^{-A_{11} \rho_r^2} (1 + A_{11} \rho_r^2 - (A_{11} \rho_r^2)^2)$$

This derivative was independently re-derived from $F(Z)$ above (via $u = \rho_r^2$ and $d(\rho_r^n)/dZ = -n\rho_r^n/Z$) and confirmed to match the code's `dc4/df` terms exactly, including sign. The module docstring notes the reference workbook ("Z factor calculation.xls") uses a different, incorrect derivative that happens to still converge (D-005); this module uses the mathematically correct one, so it and the workbook can diverge in iteration path while (per D-005) landing on the same converged root.

The 11 coefficients ($A_1 \dots A_{11}$):

A_1	A_2	A_3	A_4	A_5	A_6	A_7	A_8	A_9	A_{10}	A_{11}
0.3265	-1.07	-0.5339	0.01569	-	0.5475	-0.7361	0.1844	0.1056	0.6134	0.721
				0.05165						

Inputs & units.

Symbol	Meaning	Units
p	Pressure	psia
T	Temperature	deg R
T_{pc}, p_{pc}	Pseudocriticals	deg R, psia
tol	Newton convergence tolerance on $ F(Z) $	default 10^{-10}
max_iter	Newton iteration cap	default 100
z0	Optional warm-start Z	dimensionless

Returns Z (dimensionless).

Validity & guards. Raises InputValidationError if $p < 0$; if T , T_{pc} , or $p_{pc} \leq 0$; if $z0$ is supplied and ≤ 0 . Separately raises InputValidationError unless $1.0 < T_{pr} \leq 3.0$ **and** $p_{pr} < 30.0$; there is no enforced lower bound on p_{pr} (values near zero are accepted and converge toward the ideal-gas limit $Z \rightarrow 1$, per the module docstring’s “accepting Ppr < 0.2 as the ideal-gas limit” note - this is intentional, not an unenforced guard). Each Newton step that would drive $Z \leq 0$ is reset to 10^{-3} (a defensive safety valve). Raises ConvergenceError (with iteration count and final residual) if max_iter is exhausted without meeting tol.

Source. Dranchuk, P.M. and Abou-Kassem, J.H. (1975). Coefficients per the original paper, as stated in the module docstring.

Function signature.

```
def z_factor(p_psia: float, t_r: float, tpc_r: float, ppc_psia: float, *,
            tol: float = 1e-10, max_iter: int = 100, z0: float | None = None) -> float:
```

Anchoring. test_golden_fixture1 is a **workbook golden**: four pressures against "Z factor calculation.xls"-cached Z values at $T_{pc} = 527.028947342463$ R, $p_{pc} = 676.464314208584$ psia (SBV on the workbook’s own critical-property table), $T = 703.8$ R (the workbook’s $T_F + 460$ Rankine convention, not the engine’s usual $+459.67$, needed for parity with the cached values); tolerance $\text{abs}=2\text{e-}6$ (per the test’s comment, this reproduces to $\leq 8.2 \times 10^{-7}$ at this exact T basis). test_golden_fixture3_gravity_based is the same style of golden using gravity-form pseudocriticals. Supporting tests: test_low_pressure_limit ($Z \rightarrow 1$ as $p \rightarrow 0$), test_warm_start_agrees_with_cold ($z0$ doesn’t change the converged root), and a full set of InputValidationError / ConvergenceError guard tests, including test_z_positive_safety_valve which drives a Newton step negative to exercise the $Z \leq 0 \rightarrow 10^{-3}$ clamp.

4.2.2 Hall & Yarborough (1973)

Purpose. Solve the Hall & Yarborough equation of state for Z by Newton iteration on the reduced gas density y , using the **reciprocal** reduced temperature convention.

Equation(s). With $t = T_{pc}/T$ (note: reciprocal of the usual $T_{pr} = T/T_{pc}$) and $p_{pr} = p/p_{pc}$:

$$A = 0.06125 t p_{pr} e^{-1.2(1-t)^2}$$

$$B = t (14.76 - 9.76t + 4.58t^2)$$

$$C = t(90.7 - 242.2t + 42.4t^2)$$

$$D = 2.18 + 2.82t$$

Newton-solved for reduced density y :

$$F(y) = -A + \frac{y + y^2 + y^3 - y^4}{(1 - y)^3} - By^2 + Cy^D = 0$$

$$F'(y) = \frac{1 + 4y + 4y^2 - 4y^3 + y^4}{(1 - y)^4} - 2By + CDy^{D-1}$$

$$Z = \frac{A}{y}$$

F' is the closed-form derivative of F , symbolically verified per the module docstring, and is algebraically equivalent to the Gas_Gradient VBA CalculateZFactor kernel's derivative of the residual $F(y)/y$.

Inputs & units.

Symbol	Meaning	Units
p	Pressure	psia
T	Temperature	deg R
T_{pc}, p_{pc}	Pseudocriticals	deg R, psia
tol	Newton convergence tolerance on $ F(y) $	default 10^{-10}
max_iter	Newton iteration cap	default 60

Returns Z (dimensionless).

Validity & guards. Raises InputValidationError if $p < 0$ or if $T, T_{pc}, p_{pc} \leq 0$. Unlike `dak.z_factor`, this module does **not** enforce a (p_{pr}, T_{pr}) validity window - deliberate, per the module docstring; extreme inputs are handled by the Newton iteration's own defensive clamps: y is reset to 0.999 if a step would push it ≥ 1 , and to 10^{-6} if a step would push it ≤ 0 . Raises ConvergenceError if max_iter is exhausted without meeting tol.

D-006 (adjacent). A separate workbook (ADRIC CVD Additional_QC sheet) implements a broken variant of this equation: it uses the ordinary T_{pr} instead of the reciprocal t , omits the $\cdot t$ factor in the A term, and returns the reduced density y itself as " Z ". This module implements the canonical 1973 form per the Gas_Gradient VBA reference, not that broken variant.

Source. Hall, K.R. and Yarborough, L. (1973), per the module docstring.

Function signature.

```
def z_factor(p_psia: float, t_r: float, tpc_r: float, ppc_psia: float, *,
            tol: float = 1e-10, max_iter: int = 60) -> float:
```

Anchoring. This module's tests do not cite an external workbook golden; its primary anchor is a **cross-correlation check**: `test_agrees_with_dak_within_2pct` compares `hy_z(...)` against `dak.z_factor(...)` at six pressures for the same sweet 0.737-gravity gas pseudocriticals used in the DAK golden fixture, `rel=0.02`. `test_low_pressure_limit` checks $Z \rightarrow 1$ as $p \rightarrow 0$. `test_z_is_a_over_y_not_y` is a D-006 guard: at high pressure it asserts $Z > 0.7$ while the underlying reduced density y is small (~ 0.1), which would fail if the function returned y instead of A/y . `test_upper_clamp_safety_valve` drives $p_{pr} \approx 150,000$ (non-physical) specifically to exercise

the $y \geq 1 \rightarrow 0.999$ clamp and confirms iteration still converges. Full `InputValidationError` / `ConvergenceError` guard coverage is also present.

4.3 Bubble-Point Pressure (pvt/correlations/bubble_point/)

The package intentionally exposes no ambiguous bare `bubble_point` name at the package level (four independent correlations each define their own); call `<module>.bubble_point(...)` explicitly.

4.3.1 Standing (1947)

Purpose. Estimate bubble-point pressure from solution GOR, gas gravity, API gravity, and temperature, using Standing’s original California-crude correlation.

Equation(s).

$$a = 0.00091 T_F - 0.0125 API$$

$$p_b = 18.2 \left[\left(\frac{R_s}{\gamma_g} \right)^{0.83} 10^a - 1.4 \right]$$

`bubble_point_with_exponent(rs, gas_gravity, a)` exposes the same formula with a supplied directly (a parity hook matching the source workbook, which leaves a as a raw user-entered cell, D-007); `bubble_point(...)` is the normal entry point and computes a from T_F/API internally.

Inputs & units.

Symbol	Meaning	Units
R_s	Total solution GOR	scf/STB
γ_g	Gas specific gravity (air = 1)	dimensionless
API	Stock-tank oil API gravity	deg API
T_F	Reservoir temperature	deg F

Returns p_b in psia (floored at 0 for non-physical inputs).

Validity & guards. `bubble_point_with_exponent` returns 0.0 (no exception) if $\gamma_g \leq 0$ or $R_s \leq 0$; otherwise returns $\max(p_b, 0)$. `bubble_point` emits a `UserWarning` (message contains “outside Standing”) for any input outside Standing’s original data range: $R_s \in [20, 1425]$ scf/STB, $\gamma_g \in [0.59, 0.95]$, $API \in [16.5, 63.8]$, $T_F \in [100, 258]$ deg F. Neither function raises `InputValidationError`; out-of-range inputs only warn.

Source. Standing, M.B. (1947). *A Pressure-Volume-Temperature Correlation for Mixtures of California Oils and Gases*. Drill. & Prod. Prac., API.

Function signature.

```
def bubble_point_with_exponent(rs_scf_stb: float, gas_gravity: float, a: float) -> float: ...
def bubble_point(rs_scf_stb: float, gas_gravity: float, api: float, t_f: float) -> float: ...
def standing_bubble_point(R_scf_stb: float, gamma_g: float, T_F: float, API: float) -> float:
    """Deprecated alias; ORIGINAL argument order (R, gamma_g, T_F, API)."""
```

Anchoring. `test_golden_sheet_literal_form` is a **workbook golden**: `bubble_point_with_exponent(1000.0, 0.65, 0.0)` against "Bubble point pressure correlations.xls" cell F38 (which leaves $a = 0$ as a raw input, D-007), `rel=1e-10`. `test_computed_exponent_form` and the `TestBubblePointFormula` class are formula self-checks against an independently written reference implementation (`_standing_pb`) across several parametrized inputs, `rel=1e-9`. `test_range_warning` confirms the out-of-range `UserWarning`. Edge-case and physical-trend tests (`TestBubblePointEdgeCases`, `TestBubblePointPhysicalTrends`) and a dedicated `TestStandingBubblePointDeprecatedAlias` class (confirming the `DeprecationWarning` and reordered-argument equivalence) round out coverage.

4.3.2 Vasquez & Beggs (1980)

Purpose. Estimate bubble-point pressure from solution GOR, gas gravity, API gravity, and temperature, with a separator-gravity correction and an API-gravity-dependent coefficient switch.

Equation(s). Separator gas gravity correction to a 100 psia reference:

$$\gamma_{gs} = \gamma_g \left[1 + 5.912 \times 10^{-5} API T_{sep} \log_{10} \left(\frac{p_{sep}}{114.7} \right) \right]$$

Bubble-point pressure (Ahmed's tabulated Pb-form, the form this module implements):

$$p_b = \left[C_1 \left(\frac{R_s}{\gamma_g} \right) 10^{-C_3 API / (T_F + 460)} \right]^{C_2}$$

with $(C_1, C_2, C_3) = (27.62, 0.914, 11.172)$ for $API \leq 30$, and $(56.18, 0.842, 10.393)$ for $API > 30$.

An algebraically equivalent original 1980 Rs-form exists in the literature (inverted to give the Pb-form above), solved for R_s directly:

$$R_s = C'_1 \gamma_g p_b^{C'_2} \exp \left(\frac{C'_3 API}{T_F + 460} \right)$$

with $(C'_1, C'_2, C'_3) = (0.0362, 1.0937, 25.7240)$ for $API \leq 30$, and $(0.0178, 1.1870, 23.9310)$ for $API > 30$. The two forms agree to within about 0.5% (not bit-identical) because each published coefficient table is independently rounded; this module implements only the Pb-form.

Inputs & units.

Symbol	Meaning	Units
R_s	Solution GOR at bubble point	scf/STB
γ_g	Gas specific gravity (air = 1)	dimensionless
API	Stock-tank oil API gravity	deg API
T_F	Reservoir temperature	deg F
T_{sep}, p_{sep}	Separator temperature, pressure	deg F, psia

Returns p_b in psia; `corrected_gas_gravity` returns γ_{gs} (dimensionless).

Validity & guards. `bubble_point` raises `InputValidationError` if $R_s \leq 0$, $\gamma_g \leq 0$, or $API \leq 0$. `corrected_gas_gravity` raises `InputValidationError` if $\gamma_g \leq 0$, $API \leq 0$, or $p_{sep} \leq 0$. `bubble_point` additionally emits a `UserWarning` (message contains "outside Vasquez-Beggs") for inputs outside the original data range: $R_s \in [20, 2070]$ scf/STB, $\gamma_g \in [0.511, 1.351]$, $API \in [15.3, 59.3]$, $T_F \in [75, 294]$ deg F.

D-008. The source workbook computes the exponent term as $a = -C_3 API (T_F + 460)$ (multiplying by $(T_F + 460)$ instead of dividing), which overflows to `#NUM!`. This module divides by $(T_F + 460)$, per both published forms above.

Source. Vasquez, M. and Beggs, H.D. (1980). *Correlations for Fluid Physical Property Prediction*. JPT, June 1980. Tabulated coefficients per Ahmed, T., *Reservoir Engineering Handbook*.

Function signature.

```
def corrected_gas_gravity(gas_gravity: float, api: float, t_sep_f: float, p_sep_psia: float) -> float: ...
def bubble_point(rs_scf_stb: float, gas_gravity: float, api: float, t_f: float) -> float: ...
```

Anchoring. test_golden_gamma_gs is a **workbook golden**: corrected_gas_gravity(1.0, 30.0, 100.0, 150.0) against "Bubble point pressure correlations.xls" cell F64, rel=1e-10. test_corrected_bubble_point_magnitude checks $p_b \approx 5855$ psia at rel=0.02 (a controller-adjudicated anchor, cross-verified via both the Pb-form and the original Rs-form to 0.003% agreement). test_round_trip_against_original_rs_form is a genuine external cross-check: it feeds bubble_point()'s Pb-form output back into the independent, differently-parameterized 1980 Rs-form (different coefficients, exp() instead of $10^{(\cdot)}$) and recovers the input R_s to within the tabulated coefficients' own rounding precision (rel=5e-3 for $API \leq 30$, rel=6e-3 for $API > 30$, both branches checked). test_coefficient_switch_at_api_30 confirms the branch actually switches at $API = 30$. Full InputValidationError and range-UserWarning guard coverage is also present (test_range_warning_all_four_checks fires all four range warnings in one call).

4.3.3 Glaso (1980), SPE 8016

Purpose. Estimate bubble-point pressure via Glaso's generalized correlating number and a quadratic-in-log10 fit.

Equation(s).

$$p_b^* = \left(\frac{R_s}{\gamma_g} \right)^{0.816} \frac{T_F^{0.172}}{API^{0.989}}$$

$$\log_{10}(p_b) = 1.7669 + 1.7447 \log_{10}(p_b^*) - 0.30218 (\log_{10}(p_b^*))^2$$

$$p_b = 10^{\log_{10}(p_b)}$$

Inputs & units.

Symbol	Meaning	Units
R_s	Solution GOR at bubble point	scf/STB
γ_g	Gas specific gravity (air = 1)	dimensionless
API	Stock-tank oil API gravity	deg API
T_F	Reservoir temperature	deg F

Returns p_b^* (dimensionless, from pb_star) or p_b in psia (from bubble_point).

Validity & guards. Both pb_star and bubble_point raise InputValidationError (collecting all violations) if $R_s \leq 0$, $\gamma_g \leq 0$, $API \leq 0$, or $T_F \leq 0$. No warning-only range check is implemented for this correlation.

D-009. The source workbook multiplies p_b^* by a stray factor of 14.5 and never applies the final $10^{(\cdot)}$ step (returning $\log_{10}(p_b)$ itself, mislabeled as p_b); it also rounds the correlation constants to 1.767/1.745. This module applies neither deviation: pb_star() has no stray factor, bubble_point() exponentiates back out of log space, and the published 1.7669/1.7447 constants are used.

Source. Glaso, O. (1980). *Generalized Pressure-Volume-Temperature Correlations*. JPT, May 1980, pp. 785-795 (SPE 8016).

Function signature.

```
def pb_star(rs_scf_stb: float, gas_gravity: float, api: float, t_f: float) -> float: ...
def bubble_point(rs_scf_stb: float, gas_gravity: float, api: float, t_f: float) -> float: ...
```

Anchoring. `test_pb_star_matches_workbook_cell` is a **workbook golden**: `pb_star(1000.0, 0.65, 30.0, 200.0)` against the reference sheet's cached cell F81 = 497.662528246482, divided by the sheet's own stray 14.5 factor (D-009), $\text{rel}=1\text{e-}6$ - an exact anchor for the published P_b^* form derived by dividing out a known bug. `test_corrected_magnitude` checks $p_b \approx 5413.4$ psia at $\text{rel}=0.02$ (hand-derivation: $p_b^* = 34.32$, $\log_{10}(p_b) = 3.7336$). `test_trends` confirms the expected GOR/API/gravity directions. Full `InputValidationError` guard coverage is present for both `pb_star` and `bubble_point`.

4.3.4 Al-Marhoun (1988)

Purpose. Estimate bubble-point pressure for Middle East crude oils from solution GOR, gas gravity, stock-tank oil specific gravity, and temperature.

Equation(s).

$$T_R = T_F + 459.67$$

$$p_b = 5.38088 \times 10^{-3} R_s^{0.715082} \gamma_g^{-1.87784} \gamma_o^{3.1437} T_R^{1.32657}$$

Inputs & units.

Symbol	Meaning	Units
R_s	Solution GOR at bubble point	scf/STB
γ_g	Gas specific gravity (air = 1)	dimensionless
γ_o	Stock-tank oil specific gravity	dimensionless
T_F	Reservoir temperature	deg F

Returns p_b in psia.

Validity & guards. Raises `InputValidationError` (collecting all violations) if $R_s \leq 0$, $\gamma_g \leq 0$, $\gamma_o \notin (0, 2)$, or $T_F \leq -459.67$ (absolute zero). No warning-only range check is implemented for this correlation.

Source. Al-Marhoun, M.A. (1988). *PVT Correlations for Middle East Crude Oils*. Journal of Petroleum Technology, 40(5), 650-666.

Function signature.

```
def bubble_point(rs_scf_stb: float, gas_gravity: float, oil_sg: float, t_f: float) -> float:
```

Anchoring. `test_full_precision_value` is a formula self-check: it recomputes the equation term-by-term independently and asserts exact agreement ($\text{rel}=1\text{e-}12$). `test_close_to_sheet_rounded_form` is a **loose workbook golden**: $p_b \approx 5585.23$ psia at $\text{rel}=0.006$, matching the reference sheet's rounded-coefficient result with $T_F + 460$ (the full-precision published form used here lands within 0.6% of the sheet's rounded one). `test_trends` confirms higher GOR gives higher p_b . Full `InputValidationError` guard coverage is present, including the oil-SG bounds and the absolute-zero temperature floor.

4.4 Gas Viscosity (pvt/correlations/viscosity/)

4.4.1 Lee, Gonzalez & Eakin (1966), SPE 1340

Purpose. Estimate natural gas viscosity from temperature, apparent molecular weight, and gas density (itself computed from the real-gas law).

Equation(s).

$$K = \frac{(9.4 + 0.02 M) T^{1.5}}{209 + 19M + T} \quad (T \text{ in deg R})$$

$$X = 3.5 + \frac{986}{T} + 0.01 M$$

$$Y = 2.4 - 0.2X$$

$$\mu_g = 10^{-4} K e^{X \rho_g^Y} \quad (\rho_g \text{ in g/cc, } \mu_g \text{ in cP})$$

Gas density input, via the real-gas law:

$$\rho_g = \frac{p M}{Z T_R} \cdot \text{DENSITY_COEF}, \quad T_R = T_F + 459.67$$

where DENSITY_COEF is the exact lbm/ft³ to g/cc conversion divided by the gas constant:

$$\text{DENSITY_COEF} = \frac{G_{PER_LB}}{30.48^3} / R_{PSIA_FT3_LBMOL_R} = \frac{453.59237}{28316.846592} / 10.7316 \approx 0.0014926$$

Inputs & units.

Symbol	Meaning	Units
p	Pressure	psia
M	Gas apparent molecular weight	g/mol (lbm/lbmol)
Z	Gas compressibility factor	dimensionless
T_F	Temperature	deg F
ρ_g	Gas density (from gas_density_g_cc)	g/cc

Returns ρ_g (g/cc) or μ_g (cP).

Validity & guards. gas_density_g_cc raises InputValidationError (collecting all violations) if $p < 0$, $M \leq 0$, or $Z \leq 0$. gas_viscosity_cp raises InputValidationError if $M \leq 0$ or $\rho_g < 0$.

D-010. The source workbook (5_Viscosity_HPHT_Calc_v2.xlsx) hardcodes the lbm/ft³ to g/cc conversion as the rounded literal 0.0014935. This module derives DENSITY_COEF exactly from canonical constants instead (30.48 cm/ft is the formula-grade, exact-by-definition inch/foot conversion); the two agree to about 0.06%.

Source. Lee, A.L., Gonzalez, M.H., and Eakin, B.E. (1966). *The Viscosity of Natural Gases*. JPT, August 1966, pp. 997-1000 (SPE 1340).

Function signature.

```
def gas_density_g_cc(p_psia: float, mw: float, z: float, t_f: float) -> float: ...
def gas_viscosity_cp(t_f: float, mw: float, rho_g_cc: float) -> float: ...
```

Anchoring. test_golden_viscosity_workbook_point is a **workbook golden**: at 965 psia, $Z = 0.945$, $M = 19.5$, $T_F = 256$, the reference workbook caches $\rho = 0.04155$ g/cc and $\mu_g = 0.015395$ cP; the test checks ρ at rel=2e-3 and μ_g at rel=5e-3 (D-010's exact coefficient is about 0.06% lower than the workbook's rounded one, so the tolerance absorbs that known, intentional difference rather than masking a regression). test_viscosity_increases_with_density and test_dilute_limit_positive check physical trends/bounds. Full InputValidationError guard coverage is present for both functions.

4.4.2 Jossi, Stiel & Thodos (1962)

Purpose. Estimate dense-gas viscosity from reduced density and a dilute-gas (“zero-density”) viscosity term, transcribed from the Amoco GasProp VBA ThodosGasVisc.

Equation(s).

$$\chi = \frac{(T_{pc}/1.8)^{1/6}}{\sqrt{M} (p_{pc}/14.696)^{2/3}} \quad (T_{pc} \text{ in deg R, } p_{pc} \text{ in psia})$$

Dilute-gas term, split at $T_r = T/T_{pc} = 1.5$:

$$T_r \leq 1.5 : \quad \mu^* = \frac{0.00034 T_r^{0.888}}{\chi}$$

$$T_r > 1.5 : \quad \mu^* = \frac{0.001668 (0.1338 T_r - 0.0932)^{5/9}}{\chi}$$

Dense-gas viscosity:

$$\mu = \frac{(0.1023 + 0.023364 \rho_r + 0.058533 \rho_r^2 - 0.040758 \rho_r^3 + 0.0093324 \rho_r^4)^4 - 10^{-4}}{\chi} + \mu^*$$

Reduced density input:

$$\rho_r = \frac{V_{c,mix} p}{Z R T} \quad (R = 10.7316 \text{ psia} \cdot \text{ft}^3 / (\text{lbmol} \cdot ^\circ\text{R}))$$

Inputs & units.

Symbol	Meaning	Units
T	Temperature	deg R
M	Gas apparent molecular weight	g/mol
T_{pc}, p_{pc}	Mixture pseudocriticals	deg R, psia
ρ_r	Reduced density (from reduced_density)	dimensionless
$p, Z, V_{c,mix}$	Pressure, Z-factor, mixture critical volume	psia, dimensionless, ft ³ /lbmol

Returns ρ_r (dimensionless, from reduced_density) or μ (cP, from gas_viscosity_cp).

Validity & guards. reduced_density raises InputValidationError (collecting all violations) if $Z \leq 0$ or $V_{c,mix} \leq 0$. gas_viscosity_cp raises InputValidationError if $M \leq 0$, $T_{pc} \leq 0$, $p_{pc} \leq 0$, or $\rho_r < 0$.

Amoco-variant caveat. The $T_r \leq 1.5$ / $T_r > 1.5$ dilute-term split and its coefficients (0.00034/0.888; 0.001668/0.1338/0.0932/(5/9)) are the Amoco GasProp VBA's variant of the Stiel-Thodos (1961) dilute-gas viscosity correlation, not a transcription of the originally published Stiel-Thodos coefficients; this VBA variant deviates from published forms by roughly -2% to $+3\%$ depending on T_r . The two branches do **not** meet exactly at $T_r = 1.5$ (confirmed to differ by up to 5%, faithful to the VBA source, not a transcription error).

D-011. The VBA source hardcodes the gas constant in `reduced_density` as the rounded literal 10.73. This module uses the canonical `R_PSIA_FT3_LBMOL_R` (10.7316) instead; the two agree to about 0.015%.

Source. Jossi, J.A., Stiel, L.I., and Thodos, G. (1962). *The Viscosity of Pure Substances in the Dense Gaseous and Liquid Phases*. AIChE Journal, 8(1), 59-63. Transcribed from the preserved VBA ThodosGasVisc (docs/reference/gasprop_functions.bas).

Function signature.

```
def reduced_density(p_psia: float, z: float, t_r: float, vc_mix: float) -> float: ...
def gas_viscosity_cp(t_r: float, mw: float, tpc_r: float, ppc_psia: float, rho_r: float) -> float: ...
```

Anchoring. No external workbook golden is used for this module; its tests are **formula self-checks**. `test_zero_density_recovers_dil` recomputes χ and μ^* (the $T_r > 1.5$ branch) independently and checks the $\rho_r = 0$ result at `rel=1e-9`. `test_reduced_density_formula` recomputes the ρ_r formula directly, `rel=1e-12`. `test_branch_boundary_continuity_documented` evaluates just below and just above $T_r = 1.5$ and asserts they agree only to `rel=0.05`, explicitly documenting (not hiding) the branch discontinuity. `test_monotone_in_reduced_density` checks the expected trend. Full `InputValidationError` guard coverage is present for both functions.

4.4.3 Critical Volume Table (`critical_volumes.py`)

Purpose. Supply the mole-fraction-weighted mixture critical volume $V_{c,mix}$ that `jossi_stiel_thodos.reduced_density` consumes, using a fixed 11-component table plus a per-call C7+ value.

Equation(s).

$$V_{c,mix} = \sum_i y_i V_{c,i}$$

where $V_{c,i}$ is looked up in `VC_TABLE` for the 11 tabulated components, or supplied as the `c7_plus_vc` argument for the "C7+" key (typically `erbar.c7_plus_criticals(mw, sg)` [2], Hall's 1971 V_c).

`VC_TABLE` (ft³/lbmol), ported verbatim, in the VBA's fixed positional order:

N2	C1	CO2	C2	H2S	C3	iC4	nC4	iC5	nC5	C6
1.44	1.59	1.51	2.37	1.565	3.21	4.21	4.08	4.9	4.87	5.93

C7+ is deliberately excluded from the table; it is not a fixed constant because the VBA (and this module) computes it per-call.

Inputs & units.

Symbol	Meaning	Units
<code>mol_fractions</code>	Mole fraction (0-1) by component code	dimensionless
<code>c7_plus_vc</code>	C7+ pseudo-component critical volume (used only if "C7+" is a key)	ft ³ /lbmol

Returns $V_{c,mix}$ in ft³/lbmol.

Validity & guards. Raises InputValidationError listing any unknown component code(s) in mol_fractions (i.e. not in VC_TABLE and not "C7+").

Source. Gas_Gradient VBA CalculateCriticals (docs/reference/gasprop_functions.bas, lines ~437-444), ported verbatim; not itself a separately dated literature correlation (the values are Amoco-internal component-property constants, distinct from the Hall (1971) correlation used to compute the C7+ entry).

Function signature.

```
def vc_mix(mol_fractions: dict[str, float], c7_plus_vc: float = 0.0) -> float:
```

Anchoring. test_table_matches_vba_spot_values is an **exact VBA spot check**: every VC_TABLE entry against the .bas file's crit_vc array values, confirmed positionally against the parallel mol_wt array at the same lines. test_table_has_exactly_eleven_entries_no_c7_plus confirms the table's shape contract. test_pure_component_identity and test_pure_c7_plus_identity check single-component mixes recover the table value (or the supplied c7_plus_vc) exactly, rel=1e-12. test_mole_fraction_weighted_mix recomputes a four-component mix independently. test_unknown_key_raises / test_unknown_key_raises_alongside_known_keys cover the guard.

4.5 Summary Table

Module	Correlation	Source year	Anchor type
pseudocritical/sutton.py	Sutton	1985	Formula self-check (exact)
pseudocritical/sbv.py	Stewart-Burkhardt-Voo	1959	Workbook golden + exact identity
pseudocritical/piper_mccain.py	Piper, McCain & Corredor (SPE 26668)	1993	Workbook golden + hand-derived exact pins
pseudocritical/wichert_aziz.py	Wichert-Aziz	1972	Formula self-check (hand-derived)
pseudocritical/erbar.py	Erbar (Chao-Seader) / Hall Vc	n/a (VBA) / 1971 (Vc)	VBA line-trace (transcription self-consistency)
zfactor/dak.py	Dranchuk & Abou-Kassem	1975	Workbook golden
zfactor/hall_yarborough.py	Hall & Yarborough	1973	Cross-correlation check (vs DAK)
bubble_point/standing.py	Standing	1947	Workbook golden + formula self-check
bubble_point/vasquez_beggs.py	Vasquez & Beggs	1980	Workbook golden + cross-form round-trip
bubble_point/glaso.py	Glaso (SPE 8016)	1980	Workbook golden (exact) + hand-derived magnitude
bubble_point/almarhoun.py	Al-Marhoun	1988	Formula self-check + loose workbook golden
viscosity/lee_gonzalez_eakin.py	Lee, Gonzalez & Eakin (SPE 1340)	1966	Workbook golden
viscosity/jossi_stiel_thodos.py	Jossi, Stiel & Thodos (Amoco VBA variant)	1962	Formula self-check
viscosity/critical_volumes.py	Vc table (Gas_Gradient VBA)	n/a (VBA transcription)	VBA spot-check (exact)

Chapter 5 - Flash Separation (Atmospheric Flash, Water-Pump Method)

Module 2 of the platform. This chapter covers the single-stage atmospheric flash test: what the lab does at the bench, the three ways to get that data into the platform, the exact calculation chain the engine runs, the mass-basis recombination that turns flashed oil + flashed gas back into a whole-sample composition, the QC checks applied, and the report the test produces. Every equation below is transcribed from `pvt/experiments/flash/calc.py`, `pvt/experiments/flash/recombine.py`, and `pvt/core/plus_fractions.py`. The worked numbers are the SA-372 sample, taken from `tests/golden/test_flash_sa372.py`, `tests/golden/test_flash_recombination_sa372.py`, and `tests/fixtures/sa372_flash.py`, cached from `ADRIC_Flash_Separation_Calc_v6.1.xlsx`.

5.1 The laboratory test

The engine's own module docstring names the method precisely:

Atmospheric flash separation, water-pump method (ADRIC Flash v6.1 methodology).

A single-stage flash takes a live reservoir-oil sample and expands it, in one step, from cell/charge pressure down to atmospheric pressure and stock-tank conditions. What comes out is a stock-tank oil and a liberated gas; the volumes, masses, and compositions of both are what this test measures.

The ADRIC Flash v6.1 template's `Volumetrics_Master` sheet (the workbook the importer reads - see 5.2) lays the bench procedure out as three instrument blocks:

The displacement pump. The live-oil cylinder is metered through a positive-displacement pump into the flash line. The pump's own reading is taken before and after the transfer (`pump_initial_cc`, `pump_final_cc`, `cc`), and the metered volume is corrected by two calibration factors read off the pump itself: a **pump constant** (`pump_constant`) and a **volume correction factor** (`vcf`). This gives the charge-pressure volume of live oil that was pushed through the flash valve - equation (5.1) below.

The stock-tank flask. What is left after the oil flashes to atmospheric conditions is caught in a stock-tank flask. Its volume is read directly off the flask's graduation (`v_sto_cc`) - this is an *input*, not something the engine derives. Its mass is the flask's gross weight minus its tare weight (`oil_tare_g`, `oil_gross_g`), giving the flashed-oil mass by difference - equation (5.2).

The gasometer. Gas liberated at the flash valve is routed to a gasometer - a water-displacement gas meter. Its reading is likewise taken before and after (`gasometer_initial_cc`, `gasometer_final_cc`) and corrected by a **gasometer factor** (`gasometer_factor`) - equation (5.3). Because the gasometer reads at whatever temperature and pressure prevail in the room and the drum at the time (`gas_temp_c`, `gas_abs_pressure_mbar`), that raw volume is then corrected to lab standard conditions (14.73 psia / 60°F) with the combined gas law - equation (5.4). Gas gravity (air = 1) is measured separately (`gas_gravity`) and is not derived by this chain - it feeds the gas-density equation (5.5) directly.

Both liberated streams - the stock-tank oil and the flash gas - are also sent for GC compositional analysis against the 52-component Katz-Firoozabadi table (mol% and wt% bases for each). That composition data is what section 5.4's mass recombination and section 5.5's composition QC run on.

5.2 Data entry

The platform accepts flash data through three routes, all producing the same `FlashVolumetrics` dataclass (13 fields) that `pvt.experiments.flash.calc.calculate` consumes.

Route 1 - App upload

Upload a filled `ADRIC_Flash_Separation_Calc_v6.1.xlsx` template on the Flash Separation page's **Upload Workbook** tab. The importer (`pvt.io.excel_import.flash_v61.read`) reads the `Volumetrics_Master` sheet only - the other sheets (`Component_Properties`, `Recombination`, `Plus_Properties_Report`) are downstream/computed in the workbook and are not read. It pulls:

- **Sample metadata** (rows 5–8): Client, Well/Field, Sampling Depth, Sample ID, Chamber/Cylinder, Project No., Fluid Type.
- **Volumetric inputs** (rows 11–21, the yellow input cells only - B13, B16, B19 and the whole of rows 25–37 are computed in the sheet and are deliberately not read).
- **GC compositions** (rows 41–92): component code plus Gas Mol%, Gas Wt%, Oil Mol%, Oil Wt% for all 52 Katz-Firoozabadi rows.

Two cells - E17 (barometric pressure) and E20 (back pressure) - are read by the workbook into an unused `P_base` composite (B25) that the workbook itself never consumes downstream. The engine does not read them either; see 5.5's note and the deviations ledger entry D-017.

If the workbook is missing the `Volumetrics_Master` sheet, or its header text has shifted from the expected v6.1 layout, or any composition cell reads negative, the import is rejected with a listed `InputValidationError` rather than silently reading garbage.

Route 2 - Manual form

The **Manual Entry** tab mirrors `FlashVolumetrics`'s 13 fields directly:

Field	Label	Unit	Default	Constraint (validate.py)
<code>pump_initial_cc</code>	Initial pump reading	cc	0.0	≥ 0
<code>pump_final_cc</code>	Final pump reading	cc	0.0	$> \text{pump_initial_cc}$
<code>v_sto_cc</code>	Stock tank oil volume (<code>V_sto</code>)	cc	1.0	> 0
<code>oil_tare_g</code>	Oil tare weight	g	0.0	≥ 0
<code>oil_gross_g</code>	Final oil + tare weight	g	0.0	$> \text{oil_tare_g}$
<code>gasometer_initial_cc</code>	Initial gasometer reading	cc	0.0	≥ 0
<code>gasometer_final_cc</code>	Final gasometer reading	cc	0.0	$\geq \text{gasome-ter_initial_cc}$
<code>gas_temp_c</code>	Gas temperature	°C	20.0	$-10 < t < 60$ (exclusive)
<code>gas_abs_pressure_mbar</code>	Measured gas abs. pressure	mbar	1013.25	$500 < p < 1500$ (exclusive)
<code>gas_gravity</code>	Gas gravity (Air = 1)	-	1.0	$0.5 < g < 3.0$ (exclusive)
<code>pump_constant</code>	Pump constant	-	1.0	> 0
<code>vcf</code>	Volume correction factor (VCF)	-	1.0	> 0
<code>gasometer_factor</code>	Gasometer factor	-	1.0	> 0

These are the 8 rules `pvt.experiments.flash.validate.validate` enforces (each producing its own error message; every violation is reported, not just the first). The manual form's number-input widgets additionally clamp the three explicitly-bounded fields (temperature, pressure, gravity) 0.01 inside their exclusive bounds, so the widget itself cannot submit the excluded boundary value.

Below the volumetrics fields, an **optional** GC composition editor is seeded with all 52 Katz-Firoozabadi codes and four blank columns (Gas Mol%, Gas Wt%, Oil Mol%, Oil Wt%). Composition is genuinely optional in manual mode - a stream is only built when at least one of its two columns has a non-zero entry. Leaving it blank still gives you the 12 flash results (5.3); it skips the mass recombination, plus-fraction, and composition-QC sections (5.4–5.5), which need both streams.

Route 3 - CLI

```
python cli.py flash --workbook path/to/ADRIC_Flash_Separation_Calc_v6.1.xlsx
```

The CLI's `flash` subcommand only supports the workbook-upload route - there is no field-by-field flag interface for flash (unlike `cli.py recombine`, Chapter 6). It runs the same importer, the same calculation chain, the same mass recombination, and the same four QC checks (composition normalization $\times 4$, MW consistency $\times 2$ - Hoffman-Crump is not run from the CLI), and prints the report tables (5.6) as fixed-width text to stdout.

5.3 Calculation chain

`pvt.experiments.flash.calc.calculate` runs the following chain on a validated `FlashVolumetrics`. All twelve `FlashResults` fields are produced here, in this order.

Charge-pressure volume - the live-oil volume metered through the pump:

$$V_{press} = (V_{pump,final} - V_{pump,initial}) \times k_{pump} \times VCF \quad (5.1)$$

Flashed-oil mass - by difference, gross minus tare:

$$m_{oil} = m_{oil,gross} - m_{oil,tare} \quad (5.2)$$

Measured gas volume - raw gasometer displacement, calibration-corrected:

$$V_{gas,meas} = (V_{gasometer,final} - V_{gasometer,initial}) \times k_{gasometer} \quad (5.3)$$

Gas volume at standard conditions - ideal-gas ($Z = 1$) correction from the measured absolute pressure and temperature to the lab standard (14.73 psia / 1015.5981 mbar, 60°F / 288.7056 K):

$$V_{gas,std} = V_{gas,meas} \times \frac{P_{gas}}{P_{std}} \times \frac{T_{std}}{T_{gas}} \quad (5.4)$$

where P_{gas} is `gas_abs_pressure_mbar` and $T_{gas} = t_{gas,^{\circ}C} + 273.15$ K.

Gas density at standard conditions - from the independently-measured gas gravity and standard-conditions air density (0.0012255 g/cc):

$$\rho_{gas,std} = \gamma_{gas} \times \rho_{air,std} \quad (5.5)$$

Flashed-gas mass:

$$m_{gas} = V_{gas,std} \times \rho_{gas,std} \quad (5.6)$$

Gas-oil ratio, cc gas (std) per cc stock-tank oil:

$$GOR_{cc/cc} = \frac{V_{gas,std}}{V_{sto}} \quad (5.7)$$

and in field units (scf/bbl), using the 5.61458 ft³/bbl conversion:

$$GOR_{scf/bbl} = GOR_{cc/cc} \times 5.61458 \quad (5.8)$$

Flash formation volume factor, live-oil volume at charge pressure per unit stock-tank oil volume:

$$B_{o,flash} = \frac{V_{press}}{V_{sto}} \quad (5.9)$$

Shrinkage, the reciprocal relationship:

$$Shrinkage = \frac{V_{sto}}{V_{press}} \quad (5.10)$$

Stock-tank oil density at 60°F:

$$\rho_{oil,60°F} = \frac{m_{oil}}{V_{sto}} \quad (5.11)$$

API gravity - the house convention treats g/cc at 60°F as SG 60/60 directly:

$$API = \frac{141.5}{\rho_{oil,60°F}} - 131.5 \quad (5.12)$$

If validation fails, `calculate` raises `InputValidationError` listing every rule violated (Section 5.2) rather than computing on bad data; call it with `validate_inputs=False` only if you have already validated upstream.

Worked example - SA-372

Inputs (tests/unit/experiments/test_flash_validate.py, SA372):

Input	Value	Unit
pump_initial_cc	50.0	cc
pump_final_cc	70.8945	cc
v_sto_cc	15.7576	cc
oil_tare_g	100.0	g
oil_gross_g	113.71	g
gasometer_initial_cc	500.0	cc
gasometer_final_cc	1458.2037	cc
gas_temp_c	20.0	°C
gas_abs_pressure_mbar	1012.25	mbar
gas_gravity	1.146	(air = 1)
pump_constant, vcf, gasometer_factor	1.0	(each)

All twelve outputs (tests/golden/test_flash_sa372.py::test_sa372_flash_chain, workbook cell refs noted):

Eq.	Quantity	Value	Unit	Workbook cell
(5.1)	V_{press}	20.8945	cc	B13
(5.2)	m_{oil}	13.71	g	B16
(5.3)	$V_{gas, meas}$	958.2037	cc	B19
(5.4)	$V_{gas, std}$	940.5655	cc	B27
(5.5)	$\rho_{gas, std}$	0.001404423	g/cc	B28
(5.6)	m_{gas}	1.32095	g	B29
(5.7)	$GOR_{cc/cc}$	59.6896	cc/cc	B31
(5.8)	$GOR_{scf/bbl}$	335.13	scf/bbl	B32
(5.9)	$B_{o, flash}$	1.32600	vol/vol	B33
(5.10)	$Shrinkage$	0.754151	-	B34
(5.11)	$\rho_{oil, 60^\circ F}$	0.870056	g/cc	B36
(5.12)	API	31.133	°API	B37

5.4 Mass recombination & plus fractions

Once both the flashed-oil and flashed-gas GC compositions are available (uploaded or entered), `pvt.experiments.flash.recombine.re` blends them back into a single whole-sample (“wellstream”) composition on a **mass** basis - the live-fluid technique.

Gas and oil mass fractions, from the two measured masses (5.2, 5.6):

$$w_{f,gas} = \frac{m_{gas}}{m_{gas} + m_{oil}} \quad w_{f,oil} = \frac{m_{oil}}{m_{gas} + m_{oil}} \quad (5.13, 5.14)$$

Wellstream wt% blend - each stream’s own normalized (sum-to-100) wt% basis, mass-fraction-weighted:

$$w_{whole,i} = w_{f,gas} \cdot w_{gas,i} + w_{f,oil} \cdot w_{oil,i} \quad (5.15)$$

Mol% back-calculation - the wt% blend is the primary result; mol% is derived from it and renormalized, so the two MW routes agree by construction:

$$n_{raw,i} = \frac{w_{whole,i}}{MW_i} \quad z_{whole,i} = \frac{n_{raw,i} \times 100}{\sum_i n_{raw,i}} \quad (5.16, 5.17)$$

Whole-sample molecular weight (`mw_from_wt` on the resulting wellstream):

$$MW_{whole} = \frac{100}{\sum_i (w_{whole,i}/MW_i)} \quad (5.18)$$

Worked SA-372 numbers (`tests/golden/test_flash_recombination_sa372.py::test_golden_wf_and_mw`, Recombination sheet cells B18/B21):

$$w_{f,gas} = 0.0878821 \quad MW_{whole} = 135.0426 \text{ g/mol}$$

Plus fractions - the positional cut convention

`pvt.core.plus_fractions.plus_fraction` computes C7+, C11+, C20+, and C36+ cut properties. The cut boundary is **positional on the component library’s fixed 52-slot order, not name-pattern matching**: a cut is every component at or after its start code’s slot. Per the module docstring, this is what makes “C7+” **exclude** the cyclics that sort *before* C7 in the Katz-Firoozabadi table - MCP, Benzene, CycloC6 - while **including** the cyclics that sort *after* it - MCH,

Toluene - matching the flash workbook's `Plus_Properties_Report` convention. (Library order around the boundary: ... C6, MCP, Benzene, CycloC6, **C7**, MCH, Toluene, C8, ... - only C7 onward is in the cut.)

For a cut with mol% z_i and wt% w_i (both on the stream's normalized basis) and component codes restricted to the cut:

$$mol\%_{cut} = \sum_{i \in cut} z_i \quad wt\%_{cut} = \sum_{i \in cut} w_i \quad (5.19, 5.20)$$

$$MW_{cut} = \frac{\sum_{i \in cut} z_i \cdot MW_i}{mol\%_{cut}} \quad (5.21)$$

$$\rho_{cut} = \frac{wt\%_{cut}}{\sum_{i \in cut} (w_i / \rho_i)} \quad (5.22)$$

(5.22 is ideal-mixing density: same denominator form as (5.11)'s reciprocal build, applied per-cut.)

Worked C7+ numbers, SA-372:

Flashed oil alone (before recombination; `tests/golden/test_flash_recombination_sa372.py::test_cut_boundaries`) demonstrates the boundary directly - the flashed-liquid composition already excludes MCP/Benzene/CycloC6 from its C7+ figure:

$$mol\%_{C7+,oil} = 79.873\%$$

Recombined wellstream (`test_golden_c7_plus_of_recombined`, `Plus_Properties_Report` "Recombined" column):

Property	Value	Unit
mol%	51.119	%
wt%	84.236	%
MW	222.53	g/mol
Density	0.84661	g/cc

5.5 QC applied

QC only runs on the composition streams - it needs both an oil and a gas `CompositionStream` (5.2, Route 1 or an optional Route 2 entry). Each check runs independently: one check raising `InputValidationError` (for example, a manual-entry stream with mol% only and no wt% basis, which `mw_consistency` cannot grade) does not block the others - the page renders a caption explaining the skip and continues.

Check	Runs on	Threshold (review / fail)	What it grades
<code>composition_sum</code>	Gas mol%, Gas wt%, Oil mol%, Oil wt% (4 checks)	0.5 / 2.0 (points off 100)	Raw composition sum vs. 100 - eq. (5.23)
<code>mw_consistency_pct</code>	Gas, Oil (2 checks)	5.0% / 10.0%	mol%-derived MW vs. wt%-derived MW - eq. (5.24)
<code>hoffman_r2</code>	Gas/liquid pair (1 check)	$R^2 \geq 0.98$ / ≥ 0.95 (floor)	Hoffman-Crump K-value consistency crossplot - eqs. (5.25)–(5.26)

$$deviation_{sum} = raw_{sum} - 100.0 \quad (5.23)$$

$$mw_consistency\% = \frac{MW_{mol} - MW_{wt}}{MW_{wt}} \times 100 \quad (5.24)$$

The Hoffman-Crump crossplot (`pvt.qc.checks.hoffman_crump`) forms one point per component present, with positive mole fraction, in **both** the gas and liquid streams:

$$K_i = \frac{y_i}{x_i} \quad b_i = \frac{\log_{10}(P_{c,i}/14.7)}{1/T_{b,i} - 1/T_{c,i}} \quad F_i = b_i \left(\frac{1}{T_{b,i}} - \frac{1}{T_R} \right) \quad y\text{-axis} = \log_{10}(K_i \cdot P) \quad (5.25)$$

(P = flash pressure, T_R = flash temperature in Rankine; 14.7 psia is the Hoffman-Crump correlation's own fixed reference pressure, not one of the engine's standard-condition constants.) A least-squares line is fit ($\log_{10}(KP)$ vs. F) and graded on its R^2 ; since `grade()` only understands "smaller deviation is better," the R^2 -floor pair is converted:

$$deviation_{R^2} = 1 - R^2, \text{ graded against } (1 - 0.98, 1 - 0.95) \quad (5.26)$$

Fewer than 2 qualifying components, or a degenerate fit (all points sharing one F-factor or one $\log_{10}(KP)$), raises `InputValidationError` - the page shows a warning and skips the plot rather than crashing.

QC pills. Each `QCResult` renders as a coloured dot + check id + message (`ui.common.components.qc_pill`): green for PASS, amber for REVIEW, red for FAIL - the same three colours the report workbook fills the value cell with (38A169 / DD9A0A / E53E3E). Grading is inclusive downward: a value exactly at the fail threshold still grades REVIEW, not FAIL.

Worked SA-372 QC - every check passes cleanly on this sample:

Check	Value	Severity
Gas mol% sum	100.0000 (Δ 0.0000)	PASS
Gas wt% sum	100.0000 (Δ 0.0000)	PASS
Oil mol% sum	100.0000 (Δ 0.0000)	PASS
Oil wt% sum	100.0000 (Δ 0.0000)	PASS
MW consistency (gas)	+0.0031%	PASS
MW consistency (oil)	+0.1465%	PASS
Hoffman-Crump	$R^2 = 0.9819$, 22 points	PASS

Note (deviations ledger D-017): the workbook's `Volumetrics_Master!B25` computes a `P_base` composite from barometric pressure (E17) and back pressure (E20) but never uses it downstream; the engine likewise takes only the measured absolute pressure input (`gas_abs_pressure_mbar`) for eq. (5.4), and does not read E17/E20 at all.

5.6 Report contents

`pvt.reporting.tables.flash_tables` builds three sections from a `FlashResults`, a `MassRecombination`, and whatever QC results were run:

1. **Flash Results** - all twelve values from Section 5.3, formatted to the same precision as the golden tests (e.g. GOR to 4 decimal places cc/cc and 2 decimal places scf/bbl, API to 1 decimal).
2. **Whole Sample** - Gas Mass Fraction and Oil Mass Fraction (wt%, from 5.13/5.14) and Whole Sample MW (from 5.18).
3. **QC Summary** - one row per `QCResult` passed in: check id, severity, and the full message. Every QC result the caller runs appears here, whichever of Section 5.5's checks actually ran.

`pvt.reporting.excel_export.write_report` writes these, plus a Sample Information block (sample ID, well, field, reservoir, depth, fluid type, cylinder, client, project), to a single-sheet, navy-banner-styled `.xlsx` workbook. QC Summary value cells are fill-coloured to match the pill colours above. The download filename is prefixed with the sample ID (`report_download`, `ui.common.components`) so reports for different samples never collide in a downloads folder.

The mass-recombination report (and therefore the Whole Sample / QC Summary tables) is only available once both composition streams are present; with volumetrics only, the page shows the 12-result metric cards and calculation steps but prompts for composition before the report download appears.

The CLI's `flash` subcommand prints the same three tables as fixed-width text (no Excel file) alongside the Sample Information header.

Chapter 6 - Recombination / Live Oil Preparation

Module 1 of the platform. This chapter covers preparing a live (recombined) fluid sample for PVT cell testing: the two engineering routes the platform supports, the equations each runs, planning and verifying the cylinder charge, K-values, an open ruling on GOR-basis direction you should know about before trusting a Separator-basis molar split, and the QC/report each route produces. Equations are transcribed from `pvt/experiments/recombination/calc.py`, `models.py`, `compressibility.py`, `molar.py`, and `loading.py`. Worked numbers are the SA-372 sample from `tests/golden/test_molar_recombination_sa372.py` and `tests/golden/test_loading_sa372.py`, cached from `ADRIC_LiveOil_Preparation_Calc_v4.1.xlsx`.

6.1 Purpose

A live-oil PVT cell needs to be charged with a fluid that reproduces the reservoir fluid's gas-oil ratio - separator oil and separator gas (or stock-tank oil and recombination gas) metered together in the right proportion to hit a target GOR. Recombination is the calculation (and verification) that gets that proportion right before - and after - you charge the cylinder.

6.2 Two routes, and when to use each

6.2.1 Volumetric SF/FF (Carlsen & Whitson)

This is the platform's **Volumetric (SF/FF)** tab, implemented by `calculate_multistage`. Its module docstring states the nomenclature and governing relation directly:

Nomenclature follows Carlsen & Whitson (IPTC-19775, 2020) and the Whitson manual. Total producing GOR:

$$Rp = Rp_{sep}/SF + FF \quad (6.1)$$

where Rp_{sep} is GOR at separator in scf per STB separator oil (metered), SF is the Separator-Oil Shrinkage Factor = $V_{STO}/V_{sep,oil}$, FF is the Flash Factor (scf flash gas per STB STO - the solution GOR of the separator oil), and Rp is the total producing GOR in scf per STB STO.

Use this route when you have **measured separator-stage GOR(s)** and a **shrinkage factor**, and you're charging either the separator oil itself (Case 1) or the stock-tank oil directly (Case 2).

Important convention note, from the same docstring: in this engine, the per-stage GOR input (`SeparatorStage.R`) is already shrinkage-corrected - i.e. it is Rp_{sep}/SF , not the raw metered separator-basis GOR. So the code's actual sum is:

$$Rp_{total,cc} = \sum_{stages} R_{stage,cc} + FF_{cc} \quad [\text{cc gas std} / \text{cc STO}] \quad (6.2)$$

Case 1 - oil_source="separator": separator oil is charged; SF converts $V_{sep,oil} \rightarrow V_{STO}$ for the gas-volume calculation; no flash gas ($FF = 0$).

Case 2 - oil_source="stock_tank": stock-tank oil (fully degassed) is charged; FF is the additional stock-tank flash gas (solution GOR of the separator oil). By lab-practice simplification, **all** gas - separator stages plus flash - is loaded from the separator gas cylinder.

The recombination-gas conversion factor, cc of gas at recombination conditions per cc of gas at standard conditions:

$$factor_{recomb} = \frac{P_{std}}{P_{recomb}} \times \frac{T_{recomb,R}}{T_{std,R}} \times Z_{recomb} \quad (6.3)$$

The oil-volume conversion factor is case-dependent:

$$Bo_{sep,eff} = \begin{cases} 1/SF & \text{Case 1} \\ 1.0 & \text{Case 2} \end{cases} \quad (6.4)$$

Cylinder mix ratio - cc gas at recombination conditions per cc oil charged:

$$CMR = \frac{Rp_{total,cc} \times factor_{recomb}}{Bo_{sep,eff}} \quad (6.5)$$

The oil volume is **live-fluid-volume driven** - this guarantees exact volume balance ($V_{oil,sep} + V_{gas,total,recomb} = V_{live}$ exactly):

$$V_{oil,sep} = \frac{V_{live}}{1 + CMR} \quad (6.6)$$

$$V_{oil,STO} = \begin{cases} V_{oil,sep} \times SF & \text{Case 1} \\ V_{oil,sep} & \text{Case 2 (already STO)} \end{cases} \quad (6.7)$$

Charging-pressure compressibility correction - oil volume at the pressure it is actually charged into the cylinder at (p_{charge} , default 14.7 psia), vs. the volume at recombination pressure:

$$V_{oil,charge} = V_{oil,sep} \times \exp(c_o \times (P_{recomb} - p_{charge})) \quad (6.8)$$

c_o (oil isothermal compressibility, 1/psia) is supplied by `pvt.experiments.recombination.compressibility.effective_c_o`, either as a single constant value, or evaluated from a polynomial at the reference (charging) pressure:

$$c_o(P) = a_0 + a_1P + a_2P^2 + a_3P^3 + \dots \quad (6.9)$$

Field units only (1/psia basis) - a caller collecting SI-unit coefficients must convert them (multiply the n -th coefficient by 14.5038^{n+1}) and convert p_{ref} to psia *before* calling `effective_c_o`.

Per separator stage, the gas volumes at standard, separator, and recombination conditions:

$$V_{gas,std} = R_{stage,cc} \times V_{oil,STO} \quad (6.10)$$

$$V_{gas,sep} = V_{gas,std} \times \frac{P_{std}}{P_{stage}} \times \frac{T_{stage,R}}{T_{std,R}} \times Z_{stage} \quad (6.11)$$

$$V_{gas, recomb} = V_{gas, std} \times factor_{recomb} \quad (6.12)$$

Case 2's flash gas gets the same std → recomb conversion, and is folded into the same totals as the separator-stage gas. No SA-372-specific golden test exists for this route (the SA-372 fixtures below are the Molar route); `cli.py recombine` (Section 6.7) and `tests/test_recombination_calc.py` exercise it against a generic North-Sea-style example instead - do not treat those figures as SA-372 numbers.

6.2.2 Molar (PV=ZnRT split)

This is the platform's **Molar (composition)** tab, implemented by `pvt.experiments.recombination.molar.molar_split` and `wellstream`. Use this route when you have a lab GOR plus stock-tank-oil density/MW and gas MW (or full GC compositions for both), and you want the recombined-fluid **composition**, not just volumes - this is the LiveOil v4.1 Recombination sheet's own method.

GOR basis conversion (see Section 6.6 for the direction this divides in):

$$GOR_{eff} = \begin{cases} GOR/SF & \text{basis = SEPARATOR} \\ GOR & \text{basis = STOCK_TANK} \end{cases} \quad (6.13)$$

Converted to cc/cc at standard conditions:

$$GOR_{cc} = GOR_{eff} \times 0.178108 \quad (= C_{scf}/C_{STB}) \quad (6.14)$$

Moles of gas per cc of stock-tank oil - ideal gas, PV = ZnRT solved for n at standard conditions:

$$n_{gas} = \frac{P_{std} \times GOR_{cc}}{Z_{std} \times R \times T_{std}} \quad (6.15)$$

Moles of stock-tank oil per cc of stock-tank oil:

$$n_{oil} = \frac{\rho_{STO}}{MW_{STO}} \quad (6.16)$$

Gas/oil mole fractions of the wellstream:

$$f_{gas} = \frac{n_{gas}}{n_{gas} + n_{oil}} \quad f_{oil} = \frac{n_{oil}}{n_{gas} + n_{oil}} \quad (6.17)$$

Mass fractions and wellstream MW:

$$MW_{wellstream} = f_{gas} \cdot MW_{gas} + f_{oil} \cdot MW_{STO} \quad w_{gas} = \frac{f_{gas} \cdot MW_{gas}}{MW_{wellstream}} \quad w_{oil} = \frac{f_{oil} \cdot MW_{STO}}{MW_{wellstream}} \quad (6.18)$$

Wellstream composition blend - mole-fraction-weighted, on each stream's own normalized mol% basis:

$$z_i = f_{gas} \cdot y_i + f_{oil} \cdot x_i \quad (6.19)$$

Worked example - SA-372 molar split

Inputs (tests/golden/test_molar_recombination_sa372.py): GOR = 339.0 scf/STB (stock-tank basis), $\rho_{STO} = 0.8196$ g/cc, $MW_{STO} = 187.05$ g/mol (C36+ MW overridden to 635.0), $MW_{gas} = 26.10$ g/mol, $Z_{std} = 0.99$.

Eq.	Quantity	Value	Unit	Workbook cell
(6.14)	GOR_{cc}	60.378	cc/cc	B25
(6.15)	n_{gas}	0.00258036	mol/cc STO	B26
(6.16)	n_{oil}	0.00438162	mol/cc STO	B27
(6.17)	f_{gas}	0.370636	-	B29
(6.18)	w_{gas}	0.075937	-	B31
(6.18)	$MW_{wellstream}$	127.40	g/mol	B33

Wellstream composition, eq. (6.19), golden-asserted spot values (test_golden_wellstream_composition):

$$z_{C1} = 23.17\% \quad z_{C36+} = 2.97\% \quad \sum_i z_i = 100.0\%$$

A few more codes computed by the same function/inputs, for illustration (not independently golden-pinned, but produced by the exact engine call above): C2 $\approx$ 5.37%, C3 $\approx$ 4.74%, C7 $\approx$ 3.93%, C10 $\approx$ 3.81%, C20 $\approx$ 1.04%. The full table (all components carried by both the STO and gas streams) is what the app's Wellstream Composition table and the report both show, in Katz-Firoozabadi slot order (light ends first, matching the lab GC report layout).

6.3 Cylinder loading

`pvt.experiments.recombination.loading.plan_loading` takes a target stock-tank-oil charge volume and the molar split (6.2.2) and works out how much recombination gas, by volume at gas-cylinder load conditions, must be charged alongside it to hit the split's GOR - plus whether the combined charge fits the transfer cylinder.

Stock-tank-equivalent volume of the oil charge, correcting the load-density oil volume to a 60°F-density-equivalent volume:

$$V_{STO,equiv} = V_{oil,charge} \times \frac{\rho_{STO,load}}{\rho_{STO,60^\circ F}} \quad (6.20)$$

Moles of oil charged and moles of gas required to hit the split's GOR:

$$n_{oil} = \frac{V_{STO,equiv} \times \rho_{STO,60^\circ F}}{MW_{STO}} \quad n_{gas} = split.n_{gas/cc,STO} \times V_{STO,equiv} \quad (6.21, 6.22)$$

n_{gas} expressed as a standard-conditions gas volume:

$$V_{gas,std} = n_{gas} \times Z_{std} \times R \times \frac{T_{std}}{P_{std}} \quad (6.23)$$

The 14.73 psig $\rightarrow$ psia lab convention. Gas-cylinder load pressure is converted gauge $\rightarrow$ absolute using the *lab volumetric standard*, not the atmosphere/gas-constant standard used everywhere else psig $\rightarrow$ psia conversions happen in this codebase:

$$P_{load,psia} = P_{load,psig} + 14.73 \quad (6.24)$$

The `loading.py` module docstring is explicit about why this matters:

This is deliberate - the Loading_Volumes sheet's B16/B17 gauge→absolute formulas add the lab volumetric standard (14.73 psia), **not** psig + 14.696 (constants.P_ATM_PSIA) used elsewhere for psig→psia conversions.

Real-gas conversion factor, standard cc per cc at gas-load conditions, and the resulting gas charge volume:

$$factor_{load} = \frac{P_{load} \times Z_{std} \times T_{std}}{Z_{load} \times T_{load} \times P_{std}} \quad V_{gas,charge} = \frac{V_{gas,std}}{factor_{load}} \quad (6.25, 6.26)$$

Fits check - total charge must be $\leq 95\%$ of cylinder volume (headspace reserve for thermal expansion during recombination):

$$total_{charge} = V_{oil,charge} + V_{gas,charge} \quad fits = total_{charge} \leq 0.95 \times V_{cylinder} \quad (6.27)$$

Worked example - SA-372 loading plan

Inputs (tests/golden/test_loading_sa372.py): cylinder 1000 cc, target oil charge 150 cc, oil load 2000 psig / 75°F, gas load 5000 psig / 75°F / $Z = 0.85$, $\rho_{STO,load} = 0.885$ g/cc, against the 6.2.2 split above.

Eq.	Quantity	Value	Unit	Workbook cell
-	$V_{oil,charge}$	150.0	cc	B22
(6.20)	$V_{STO,equiv}$	161.97	cc	B23
(6.21)	n_{oil}	0.709687	mol	B25
(6.22)	n_{gas}	0.417938	mol	B29
(6.23)	$V_{gas,std}$	9779.46	cc	B30
(6.25)	$factor_{load}$	385.39	std cc/cc	B31
(6.26)	$V_{gas,charge}$	25.38	cc	B32
(6.27)	fits / utilization	True / 17.5%	-	-

6.4 Actual-GOR verification

verify_actual_gor runs the same real-gas bookkeeping in reverse: given the oil and gas volumes **actually metered** into the cylinder, it recovers the as-loaded GOR and grades its deviation from the target GOR.

$$n_{actual} = \frac{V_{gas,actual} \times P_{load}}{Z_{load} \times R \times T_{load}} \quad (6.28)$$

$$V_{std} = n_{actual} \times Z_{std} \times R \times \frac{T_{std}}{P_{std}} \quad (6.29)$$

$$V_{STO,actual} = V_{oil,actual} \times \frac{\rho_{STO,load}}{\rho_{STO,60^\circ F}} \quad (6.30)$$

$$GOR_{actual} = \frac{V_{std}}{V_{STO,actual}} \times \frac{C_{STB}}{C_{scf}} \quad [\text{scf/STB}] \quad (6.31)$$

$$deviation\% = \frac{GOR_{actual} - GOR_{target}}{GOR_{target}} \times 100 \quad (6.32)$$

QC gate: gor_actual_vs_target_pct, review at >5%, fail at >10% of absolute deviation.

Worked example - SA-372, actual charge vs. plan

The plan (Section 6.3) called for charging **150 cc oil and 25.38 cc gas**. The technician's actually-metered charge came in different: **108.96 cc oil, 27.47 cc gas** (tests/golden/test_loading_sa372.py::test_golden_actual_gor_fails_gate):

Eq.	Quantity	Value
(6.31)	GOR_{actual}	505.2 scf/STB (B47)
(6.32)	$deviation\%$	+49.03% (B49)
gate	Severity	FAIL (>10%, B50)

This is the platform doing exactly the job it should: the actual charge undershot the planned oil volume and overshot the gas, and the back-calculated GOR flags it clearly rather than letting a mis-charged cylinder go to the PVT cell unnoticed.

6.5 K-values

`pvt.experiments.recombination.molar.k_values` computes equilibrium K-values from any gas/liquid stream pair, on each stream's normalized mol% basis:

$$K_i = \frac{y_i}{x_i} \quad \text{for every } i \text{ with } x_i > 0 \quad (6.33)$$

A component present only in the gas stream (x_i absent/zero) is **omitted** - K would be undefined/infinite. A component present in the liquid but absent from the gas is **included** with $K_i = 0.0$ (its y_i is treated as 0, and $x_i > 0$ still qualifies it). From the module's own unit test (tests/unit/experiments/test_molar_recombination.py): gas {C1: 80%, C2: 20%}, liquid {C2: 50%, C7: 50%} gives $K_{C2} = 0.4$, $K_{C7} = 0.0$, and C1 is excluded entirely.

This is the same y/x concept the Hoffman-Crump crossplot (Chapter 5, eq. 5.25) builds on for a flash gas/liquid pair. As of this writing, `k_values` is an engine-level utility - it is **not yet wired into the Recombination page or the report tables**; use it directly (Python/CLI script) against your gas and STO/liquid CompositionStreams if you need a K-value table for a recombination pair.

6.6 The D-018 open ruling - GOR basis direction

There is an unresolved discrepancy between the LiveOil v4.1 workbook and this engine's `GorBasis` convention, tracked as **D-018** in docs/excel-deviations.md, status proposed - NEEDS SWEJ RULING. Both readings, stated honestly:

What the workbook does. LiveOil v4.1 `Recombination!B8` is `=IF(B6="Stock Tank", B5/B7, B5)` - it divides the GOR by the shrinkage factor **on the Stock Tank branch**, and uses the raw GOR as-is on the Separator branch.

What the engine does. Eq. (6.13) above divides on the **opposite** branch: `GorBasis.SEPARATOR` divides by shrinkage (the conventional direction - a separator-metered GOR in scf per separator barrel is converted to a stock-tank basis by dividing by $SF = V_{STO}/V_{sep,oil}$); `GorBasis.STOCK_TANK` is used as-is, since it is already on a stock-tank basis.

The engine's `molar.py` module docstring calls this out explicitly at the point of decision:

D-018: conventional direction - separator-basis GOR (scf/sep-bbl) is divided by shrinkage to convert to a stock-tank (scf/STB) basis; stock-tank-basis GOR is already on that basis and used as-is. LiveOil v4.1 `Recombination!B8` implements the reverse (divides on the `STOCK_TANK` branch instead).

Why the SA-372 goldens in this chapter are unaffected either way. The SA-372 fixture's shrinkage factor is $SF = 1.0$ (tests/fixtures/sa372.py, workbook `Recombination!B7`). Dividing 339.0 by 1.0 gives 339.0 regardless of which basis branch does the dividing - the two conventions are numerically identical at $SF = 1.0$ and only diverge once $SF \neq 1$. `test_separator_basis_direction()` demonstrates this directly with a non-unity shrinkage

(0.8): `GorBasis.SEPARATOR` gives $339.0/0.8 = 423.75$ while `GorBasis.STOCK_TANK` gives 339.0 - a real, material difference once shrinkage departs from 1.0.

What this means for you. If you import a LiveOil v4.1 workbook whose `Recombination!B6` is set to “Separator” and whose shrinkage factor is not 1.0, this platform’s molar split will **not** match that workbook’s own B8 cell - by design, pending Swej’s ruling on which direction is actually correct. Until that ruling lands, treat a Separator-basis GOR with $SF \neq 1$ as a case to sanity-check by hand.

6.7 QC and report

Molar route. When compositions are available (workbook upload only - manual entry collects no GC composition), `composition_normalization.check` runs on the STO and gas streams’ mol% bases (same eq. 5.23 as Chapter 5). `mw_consistency` is **not** run here: the LiveOil v4.1 importer only reads the Mol% (INPUT) column (col I) - Wt% (INPUT) is never consumed - so these streams never carry a wt% basis to check consistency against. The Actual-GOR verification (6.4) contributes its own `gor_actual_vs_target_pct` QCResult once you submit actual charge volumes.

Volumetric SF/FF route. No QC checks are currently wired into this tab - there is no composition data in this flow to check. The self-consistency signal it does offer is `GOR_check`, a back-calculation of the total GOR from all gas actually placed in the cell (separator-stage + flash), shown alongside the input total; `cli.py recombine` prints the deviation between the two directly (flagged with a warning glyph past 0.1%), but it is not graded through `ThresholdRegistry` the way the checks above are.

Report contents.

- Molar route: `pvt.reporting.tables.recombination_tables` - **Molar Split** (9 rows, Section 6.2.2’s outputs), **Loading Plan** (10 rows, Section 6.3’s outputs plus fits/utilization), **QC Summary** (one row per QCResult run, including the Actual-GOR pill once verified).
- Volumetric route: built directly from `MultiStageResults` fields - **Setup** (live fluid volume, oil source/Case, SF, FF), **Recombination Conditions** (P/T/Z/factor_recomb), **Charge Volumes** (separator oil, charging-pressure oil, STO-equivalent, total gas at std/recomb, cylinder mix ratio), **GOR Verification** (input vs. back-calculated), **Stage GORs** (per stage). This route has no QC Summary table (nothing is graded here yet).

CLI. `cli.py recombine` runs the Volumetric SF/FF route from typed flags (stages, pressures, temperatures, Z-factors, SF/FF, recombination conditions, units field/SI, plus an optional Standing bubble-point estimate) and prints a fixed-width report. There is currently **no CLI subcommand for the Molar/LiveOil-workbook route** - unlike Chapter 5’s `cli.py flash`, there is no `cli.py recombine-molar` (or similar) that imports a filled `ADRIC_LiveOil_Preparation_Calc_v4.1.xlsx` and prints its molar-split/loading-plan report; that route is currently app-only (or direct Python calls to `pvt.io.excel_import.liveoil_v41.read + molar_split + plan_loading`).

Chapter 7: Excel Import

7.1 The Template-Import Philosophy

The PVT Lab Platform does not re-implement the ADRIC lab workbooks in Excel form and it does not trust any formula result already sitting in a cell. When a lab technician uploads a filled template, the importer opens the workbook with `openpyxl.load_workbook(path, data_only=True, read_only=True)` and reads back **only the yellow input cells**, the raw numbers and text a person typed in. Every computed quantity, from the flash gas volume at standard conditions to the wellstream molecular weight, is recalculated from scratch by the Python engine (`pvt.experiments.*`).

This gives the workbook a single job: it is the data carrier, the audited paper trail of what was measured in the lab. It is deliberately not treated as a calculator. Three consequences follow directly from the importer source:

- Cells that hold formulas or workbook-computed intermediates (block C of the Flash v6.1 Volumetrics_Master sheet, for example) are never read, even though `data_only=True` would hand back their cached values.
- If a workbook's formula logic has a known defect (see `docs/excel-deviations.md`), that defect cannot leak into a report, because the defective cell is never consulted.
- The importer fails loudly (`InputValidationError`) rather than silently, whenever the sheet it opens does not look like the template it expects, or a required cell is blank.

Both importers live under `pvt/io/excel_import/` and each exposes one function, `read(path)`, returning a frozen dataclass (`FlashImport`, `LiveOilImport`) ready to hand straight to the matching `pvt.experiments` calculation chain.

7.2 Supported Templates

Template	File	Importer module	Sheets read
ADRIC Flash Separation Calc v6.1	<code>ADRIC_Flash_Separation_Calc_v6.1.xlsx</code>	<code>pvt.io.excel_import.flash_v61</code>	<code>Volumetrics_Master</code> only
ADRIC LiveOil Preparation Calc v4.1	<code>ADRIC_LiveOil_Preparation_Calc_v4.1.xlsx</code>	<code>pvt.io.excel_import.liveoil_v41</code>	<code>Sample_Info</code> , <code>STO_Composition</code> , <code>Gas_Composition</code> , <code>Recombination</code> , <code>Loading_Volumes</code>

The Flash v6.1 workbook carries all of its yellow inputs on one sheet; `Component_Properties`, `Recombination`, and `Plus_Properties_Report` on that workbook are downstream/computed and are not opened by the importer at all. The LiveOil v4.1 workbook spreads its inputs across five sheets, all of which are opened and validated.

7.3 ADRIC Flash v6.1 Cell Map

Source: `pvt/io/excel_import/flash_v61.py`. All cells below are on the `Volumetrics_Master` sheet.

Block A: Project & Sample Information (rows 5-8)

Cell	Field	Meaning	Units
B5	Sample.client	Client	text
B6	Sample.well, Sample.field_name	Well / Field, split on " / "	text
B7	Sample.depth_ft_md	Sampling depth (MD)	ft
H5	Sample.sample_id	Sample ID	text
H6	Sample.cylinder	Chamber / cylinder	text
H8	Sample.project	Project number	text
E8	Sample.fluid_type	Fluid type	text

Sample.reservoir has no source cell in this template (only reservoir *temperature/pressure* exist, which are conditions, not a reservoir name) and is set to "".

Block B: Volumetric Measurements (rows 11-21)

Cell	Field	Meaning	Units
B11	FlashVolumetrics.pump_constant	Pump constant	dimensionless
E11	FlashVolumetrics.vcf	Volume correction factor (VCF)	dimensionless
B12	FlashVolumetrics.pump_initial	Initial pump reading	cc
E12	FlashVolumetrics.pump_final	Final pump reading	cc
B14	FlashVolumetrics.v_sto_c	Stock tank oil volume, V_sto	cc
B15	FlashVolumetrics.oil_tare	Oil tare weight	g
E15	FlashVolumetrics.oil_gross	Singl oil + tare weight	g
B17	FlashVolumetrics.gasometer_factor	Gasometer factor	dimensionless
B18	FlashVolumetrics.gasometer_initial	Initial gasometer reading	cc
E18	FlashVolumetrics.gasometer_final	Final gasometer reading	cc
B20	FlashVolumetrics.gas_temp	Gas temperature	degC
B21	FlashVolumetrics.gas_abs_meas	Measured gas absolute pressure	mbar
E21	FlashVolumetrics.gas_grav	Gas gravity (Air = 1)	dimensionless
E17	not read	Barometric pressure	mbar
E20	not read	Back pressure	mbar

E17 and E20 are read visually by the lab technician but never consumed by the engine; see section 7.9.

Block D: GC Compositions (row 40 header, rows 41-92, 52 components)

Cell (per row 41-92)	Field	Meaning	Units
B{row}	component code	Component code (aliased if needed, see 7.5)	text
E{row}	gas_stream.mol_pct[code]	Gas Mol% (INPUT)	mol%
F{row}	gas_stream.wt_pct[code]	Gas Wt% (INPUT)	wt%
G{row}	oil_stream.mol_pct[code]	Oil Mol% (INPUT)	mol%
H{row}	oil_stream.wt_pct[code]	Oil Wt% (INPUT)	wt%

Row order matches pvt.core.components.KATZ_FIR00ZABADI.codes exactly except for the three aliased codes.

7.4 ADRIC LiveOil v4.1 Cell Map

Source: pvt/io/excel_import/liveoil_v41.py. All five required sheets are opened: Sample_Info, STO_Composition, Gas_Composition, Recombination, Loading_Volumes.

Block A: Recombination Inputs (Recombination!B5:B12)

Cell	Field	Meaning	Units
B5	gor	GOR	scf/STB
B6	gor_basis	GOR type (“Separator” / “Stock Tank” dropdown text, mapped verbatim to GorBasis)	enum
B7	shrinkage	Shrinkage factor, B_st	dimensionless
B12	z_std	Z-factor at standard conditions	dimensionless

Block B: STO Properties (STO_Composition!B5, D65)

Cell	Field	Meaning	Units
B5	sto_density_60f	STO density at 60 degF, reported	g/cc
D65	c36_mw	C36+ molecular weight (editable, row 65)	g/mol

Block C: Compositions (STO_Composition!, Gas_Composition!, row 14 header, rows 15-65, 51 components)

Cell (per row 15-65)	Field	Meaning	Units
B{row}	component code	Component code (aliased if needed, see 7.5)	text
I{row}	sto_stream.mol_pct[code] / gas_stream.mol_pct[code]	Mol% (INPUT), the only composition column read	mol%
J{row}	not read	Wt% (INPUT); not consumed by any calc.py in this phase	wt%

Block D: Loading (Loading_Volumes!B5:B12)

Cell	Field	Meaning	Units
B5	LoadingInputs.cylinder_volume	Cylinder volume	cc
B6	LoadingInputs.target_oil_target	Target oil volume	cc
B7	LoadingInputs.oil_load_pressure	Oil-cylinder gauge pressure at loading	psig
B8	LoadingInputs.oil_load_temperature	Oil-cylinder temperature at loading	degF
B9	LoadingInputs.gas_load_pressure	Gas-cylinder gauge pressure at loading	psig

Cell	Field	Meaning	Units
B10	LoadingInputs.gas_load_temp	Gas-cylinder temperature at loading	degF
B11	LoadingInputs.z_gas_load	Gas Z-factor at gas-load conditions	dimensionless
B12	LoadingInputs.sto_density	STO density at oil-load conditions	g/cc
B13	not read	BSW / water content of loaded oil	%

Block E: Sample Metadata (Sample_Info!B5:B9, E5:E9)

Cell	Field	Meaning	Units
B5	Sample.client	Company / client	text
B6	Sample.well	Well	text
B7	Sample.sample_id	Sample ID	text
B8	Sample.depth_ft_md	Sampling depth (MD)	ft
B9	Sample.fluid_type	Fluid type	text
E5	Sample.field_name	Field	text
E6	Sample.reservoir	Reservoir	text
E7	Sample.cylinder	Chamber / cylinder number	text
E8	not read	Sampling date	date
E9	Sample.project	Project number	text

Unlike the Flash v6.1 importer, this template carries a Reservoir field (E6) directly, so Sample.reservoir is populated here rather than left as "".

7.5 Component Name Alias Handling

Both templates' GC composition rows use a Code column that mostly, but not exactly, matches pvt.core.components.KATZ_FIR00ZABAD. Each importer carries a small local _ALIAS dict, applied as `code = _ALIAS.get(raw_code, raw_code)` before the row is stored:

Flash v6.1 (flash_v61._ALIAS, 3 entries):

Workbook code	Library code	Workbook Component column text
Cyclohex	CycloC6	Cyclohexane
MPXylenes	MP-Xylene	M/P-Xylenes
OXylene	O-Xylene	O-Xylene

LiveOil v4.1 (liveoil_v41._ALIAS, 4 entries):

Workbook code	Library code	Workbook Component column text
Neo-C5	NeoC5	Neo-Pentane
Cyclohex	CycloC6	Cyclohexane
E-Benzene	EBenzene	E-Benzene
M/P-Xylene	MP-Xylene	M/P-Xylene

0-Xylene needs no alias in the LiveOil template; it is already spelled exactly like the library code there, unlike Flash's 0Xylene.

The LiveOil v4.1 composition block is also one component short: it carries 51 rows against the library's 52 codes, omitting TMB124 (1,2,4-Trimethylbenzene) entirely. That component is not aliased to anything, it is simply absent from every row read, matching what the lab actually reported. `CompositionStream` does not require every library code to be present, so the resulting stream is silently missing that one component.

7.6 The C36+ Molecular Weight Override

KATZ_FIR00ZABADI's built-in C36+ default molecular weight is 636.4 g/mol (see docs/excel-deviations.md D-001). The LiveOil v4.1 importer overrides this per sample: it reads `ST0_Composition!D65` as `c36_mw`, then builds

```
library = KATZ_FIR00ZABADI.with_c36_mw(c36_mw)
```

before either composition stream (`ST0_Composition`, `Gas_Composition`) is constructed. Both streams share that one per-sample library instance. This override is required for accuracy: building the streams against the unmodified canonical library reproduces the reference `f_gas` golden only to about 1e-4 (borderline); overriding C36+ MW first tightens that to about 1e-6.

The Flash v6.1 importer has no equivalent override; it always uses the canonical KATZ_FIR00ZABADI library as-is.

7.7 Wrong-File Detection

Each importer checks that the workbook it was handed is actually an instance of the template it expects, before reading any input cell.

Flash v6.1 (`flash_v61._check_layout`): the sheet must be named `Volumetrics_Master`; a missing sheet raises `InputValidationError` immediately:

```
{path}: missing required sheet 'Volumetrics_Master' – not an ADRIC Flash Separation Calc v6.1 workbook
```

Five anchor cells are then compared against expected text (A1, B40, E40, F40, G40, H40); any mismatch raises one error per mismatched cell, e.g.:

```
Volumetrics_Master!A1: expected 'ATMOSPHERIC FLASH SEPARATION – WATER PUMP METHOD (v6.1)', found <actual value>
```

LiveOil v4.1 (`liveoil_v41._check_layout`): all five required sheets (`Sample_Info`, `ST0_Composition`, `Gas_Composition`, `Recombination`, `Loading_Volumes`) must be present; any missing sheet raises `InputValidationError` naming the missing set. Nine anchor cells across those sheets are then compared against expected title/header text. The Flash v6.1 workbook is rejected by the missing-sheet check alone (it has no `Sample_Info/ST0_Composition/Gas_Composition/Loading_Volumes` sheet), but it does carry a sheet literally named `Recombination` with an entirely different layout, which is why the title-anchor check on that sheet is kept too, rather than relying on sheet-name presence alone.

A third, LiveOil-specific check validates `Recombination!B6`'s GOR-basis dropdown text against the two recognized values ("Separator", "Stock Tank"); an unrecognized value raises its own typed error naming the cell:

```
Recombination!B6: unrecognized GOR basis '<value>'
```

7.8 Blank-Cell and Negative-Composition Error Behavior

Both importers apply the same two guards, using the same helper pattern (`_num`), and both name the offending cell in the raised error.

Required scalar cells (every cell read via `_num`: volumetrics inputs, sampling depth, STO density, C36+ MW, loading inputs, recombination inputs) are read with:

```
def _num(ws, addr, errors):
    value = ws[addr].value
    if not isinstance(value, int | float):
        errors.append(f"{ws.title}!{addr} is blank or non-numeric")
        return 0.0
    return float(value)
```

A blank or non-numeric required cell is collected into errors rather than crashing `float(None)/float("...")` with a raw `TypeError/ValueError`. All such cells for a given block are checked before `InputValidationError(errors)` is raised, so a single import attempt reports every bad cell in that block at once, e.g.:

Volumetrics_Master!B14 is blank or non-numeric

Composition cells (Flash v6.1 columns E-H, rows 41-92; LiveOil v4.1 column I, rows 15-65) get a different treatment for blanks versus negatives:

- A blank or non-numeric composition cell is treated as absent, the same as an explicit zero, rather than crashing the sign check (`None < 0` raises `TypeError`).
- A composition cell that reads as a genuine negative number raises `InputValidationError`, naming the row, component code, and value, e.g.:

Volumetrics_Master!row 47 (C1): negative Gas Mol% value -0.5

Recombination!row 22 (nC5): negative Mol% value -1.2

(LiveOil errors are reported against whichever sheet, `STO_Composition` or `Gas_Composition`, the negative cell was found on; the example above uses the sheet's `.title`.)

`CompositionStream` itself does not validate value sign, so the Excel-import boundary is the deliberate place to reject a malformed lab entry sheet.

7.9 What Is Deliberately Not Read

Every importer reads a strict subset of its template's cells. The rest fall into two categories:

Computed/downstream cells, left alone because the engine recomputes them independently:

- Flash v6.1 `Volumetrics_Master`: B13, B16, B19 (computed sub-results inline in Block B), and all of Block C (B25-B37), are never read.
- Flash v6.1: `Component_Properties`, `Recombination`, and `Plus_Properties_Report` sheets are never opened at all.
- LiveOil v4.1: `STO_Composition!J{row}` and `Gas_Composition!J{row}` (Wt% INPUT column) are never read; only the Mol% column (I) is consumed.
- LiveOil v4.1: `Loading_Volumes!B13` (BSW / water content) and `Sample_Info!E8` (sampling date) have no corresponding dataclass field and are not read.

Dead cells with a known workbook defect, documented in `docs/excel-deviations.md` D-017: Flash v6.1 `Volumetrics_Master!E17` (barometric pressure) and `E20` (back pressure) feed the workbook's own B25 `P_base` composite formula (barometric + back-pressure), but that composite is never actually used downstream in the workbook, `V_gas_std` there is computed from the measured absolute pressure cell (B21) instead. The engine mirrors this: `calculate()` takes only the measured absolute pressure input (`gas_abs_pressure_mbar`, from B21) and never consumes a barometric or back-pressure value. E17 and E20 are read by eye by the lab technician for QA purposes but have no path into any calculation, in the workbook or in the engine.

Chapter 8: QC Reference

8.1 The QC Engine Model

Every QC check in the platform speaks the same small vocabulary, defined once in `pvt/qc/engine.py` and reused by every check module under `pvt/qc/checks/` and by the two QC checks that live inline in `pvt.experiments` (`gor_actual_vs_target`, see 8.5).

Severity

```
class Severity(enum.StrEnum):
    PASS = "PASS"
    REVIEW = "REVIEW"
    FAIL = "FAIL"
```

Ordered best to worst: `PASS < REVIEW < FAIL`.

QCResult

```
@dataclass(frozen=True)
class QCResult:
    check_id: str
    severity: Severity
    value: float | None
    threshold: str
    message: str
```

One `QCResult` is the outcome of one check on one stream/pair. `check_id` identifies which threshold band was used (it is also the key into `ThresholdRegistry`), `value` carries the graded numeric deviation, `threshold` is a human-readable rendering of the band that was applied, and `message` is the sentence a report or UI pill shows verbatim.

Grading a value against a two-band threshold

`grade()` is the shared banding function every check calls:

```
def grade(value, review_at, fail_at, *, absolute=True):
    graded = abs(value) if absolute else value
    if graded <= review_at:
        return Severity.PASS
    if graded <= fail_at:
        return Severity.REVIEW
    return Severity.FAIL
```

$$severity(v) = \begin{cases} \text{PASS} & |v| \leq review_at \\ \text{REVIEW} & review_at < |v| \leq fail_at \\ \text{FAIL} & |v| > fail_at \end{cases}$$

Band edges are inclusive downward: a value exactly at `fail_at` grades REVIEW, not FAIL.

Rolling many results up to one verdict

```
def worst(results: Iterable[QCResult]) -> Severity
```

Returns the highest-severity result across a collection, or PASS if the collection is empty. This is how a study-level verdict is derived from a list of individual QCResults.

ThresholdRegistry, overrides, and the audit trail

ThresholdRegistry holds one (`review_at`, `fail_at`) pair per `check_id`. It is constructed with the ADRIC house-convention defaults (ThresholdRegistry.DEFAULTS, reproduced in full in 8.6) already loaded, and any check may be overridden per study:

```
def override(self, check_id, review_at, fail_at, note) -> None:
    self._thresholds[check_id] = (review_at, fail_at)
    self.audit.append(
        f"{check_id}: threshold overridden to ({review_at}, {fail_at}) - {note}"
    )
```

Every override is appended to `registry.audit` with the caller-supplied note, so a QC report can explain why a particular study's bands differ from the house defaults, rather than silently applying a different number. `registry.get(check_id)` returns the current (`review_at`, `fail_at`) pair, whether default or overridden.

8.2 composition_normalization

Module: `pvt/qc/checks/composition_normalization.py`. Check ID: `composition_sum`.

Lab compositions (mol% or wt%) are expected to sum to 100 before use; small deviations come from analytical rounding, larger ones flag a transcription or unit error. The check grades the raw-sum deviation:

```
def check(stream, basis, registry=None) -> QCResult:
    raw_sum = stream.raw_mol_sum() if basis == "mol" else stream.raw_wt_sum()
    deviation = raw_sum - 100.0
    severity = grade(deviation, review_at, fail_at)
```

`basis` selects `stream.raw_mol_sum()` or `stream.raw_wt_sum()` (both defined on `CompositionStream` in `pvt/core/composition.py`). The signed deviation is preserved in `value/message` (so a report can tell over-from under-normalized), but `grade()` bands on its magnitude.

Default threshold band: **review at 0.5, fail at 2.0** (percentage points away from 100).

8.3 mw_consistency

Module: `pvt/qc/checks/mw_consistency.py`. Check ID: `mw_consistency_pct`.

A composition stream carrying both a mol% and a wt% basis implies two independent routes to the mixture molecular weight. This check grades how well they agree:

$$mw_consistency_pct = \frac{MW_{mol} - MW_{wt}}{MW_{wt}} \times 100$$

where $MW_{mol} = \text{stream.mw_from_mol}() (\sum z_i \cdot MW_i / \sum z_i)$ and $MW_{wt} = \text{stream.mw_from_wt}() (100 / \sum (w_i / MW_i))$, on the normalized wt% basis), both from `pvt/core/composition.py`.

```
def check(stream, registry=None) -> QCResult:
    pct = stream.mw_consistency_pct()
    severity = grade(pct, review_at, fail_at)
```

If a stream carries only one basis (mol% or wt%, not both), `mw_from_mol()/mw_from_wt()` raises `InputValidationError` and this check cannot run at all; see 8.7 for how the UI handles that.

Default threshold band: **review at 5.0%, fail at 10.0%**.

8.4 hoffman_crump

Module: `pvt/qc/checks/hoffman_crump.py`. Check ID: `hoffman_r2`.

For every component present, with a positive mole fraction, in **both** the gas and liquid streams of a flash, the classic Hoffman (1953) / Crump K-value consistency plot forms one point. A thermodynamically consistent K-value set falls on (nearly) a single straight line across all components; this check fits that line by ordinary least squares and grades its R^2 .

The b-factor and F-factor equations

For each qualifying component i , with vapor mole fraction y_i and liquid mole fraction x_i :

$$K_i = \frac{y_i}{x_i}$$

$$b_i = \frac{\log_{10}(P_{c,i}/14.7)}{1/T_{b,i} - 1/T_{c,i}}$$

$$F_i = b_i \left(\frac{1}{T_{b,i}} - \frac{1}{T_R} \right)$$

$$y\text{-axis value} = \log_{10}(K_i \cdot P)$$

where $T_{b,i}$, $P_{c,i}$, $T_{c,i}$ are the component's boiling point, critical pressure, and critical temperature (from the Component records in the active ComponentLibrary), and T_R is the flash temperature in Rankine (`u.f_to_r(t_f)`). F_i (the code's `f_factor`) is the crossplot x-axis; $\log_{10}(K_i P)$ (the code's `log10_kp`) is the y-axis. The engine implementation:

```
b = math.log10(component.pc_psia / _HOFFMAN_P_ATM) / (
    1.0 / component.tb_r - 1.0 / component.tc_r
)
f_factor = b * (1.0 / component.tb_r - 1.0 / t_r)
log10_kp = math.log10(k * p_psia)
```

The 14.7 Hoffman convention constant

`_HOFFMAN_P_ATM = 14.7`

This is the reference pressure baked into the classic b-factor correlation. It is deliberately **not** `pvt.core.constants.P_STD_PSIA` (14.73, the ADRIC lab volumetric standard) and **not** `P_ATM_PSIA` (14.696, the gas-constant / psig-to-psia basis); it is the older, rounder reference pressure the original correlation was published with. Using either engine constant in its place would shift every b-factor by a fraction of a percent, so `_HOFFMAN_P_ATM` is kept module-local rather than added to `pvt.core.constants`, it belongs to this one correlation, not to the engine's general unit system.

R-squared grading, and the pending-calibration note

`ThresholdRegistry.get("hoffman_r2")` returns an **R-squared floor** pair (`review_r2`, `fail_r2`), the default (0.98, 0.95) reads “R-squared ≥ 0.98 is PASS, R-squared ≥ 0.95 is REVIEW, below that is FAIL”, because that is how an engineer specifies acceptance on this plot. `grade()` itself only understands “smaller deviation is better” bands, so `check()` converts:

```
deviation = 1.0 - r_squared
severity = grade(deviation, review_at=1.0 - review_r2, fail_at=1.0 - fail_r2)
```

The default thresholds (0.98, 0.95) are, unlike every other entry in `ThresholdRegistry.DEFAULTS`, **not** transcribed from an ADRIC house convention. The source PVT-check sheets show this crossplot visually with no numeric R^2 gate of their own, so this default is proposed by engineering judgment, configurable via `registry.override`, pending Swej calibration.

The fewer-than-2-component typed error

Fitting a line needs at least 2 points. `_fit_least_squares` raises `InputValidationError` (not a raw `ZeroDivisionError`) in three degenerate cases:

- Fewer than 2 qualifying components (present, with a positive mole fraction, in both streams):
Hoffmann-Crump QC needs at least 2 components present in both streams (found <n>)
- All qualifying components share the same F-factor (`ss_xx == 0`, the slope’s denominator):
Hoffmann-Crump QC: all qualifying components share the same F-factor (degenerate fit, cannot compute a slope)
- All qualifying components share the same $\log_{10}(K \cdot P)$ (`ss_tot == 0`, the R^2 denominator):
Hoffmann-Crump QC: all qualifying components share the same $\log_{10}(K \cdot P)$ (degenerate fit, cannot compute R^2)

`check()` returns a `HoffmanResult` (points, slope, intercept, `r_squared`, `qc`) rather than a bare `QCResult`, so callers can also plot the fitted line.

8.5 gor_actual_vs_target

This check is **not** a module under `pvt/qc/checks/`; it is implemented inline inside `pvt.experiments.recombination.loading.verify` (`pvt/experiments/recombination/loading.py`), using the same `QCResult/grade/ThresholdRegistry` contract as every other check. Check ID: `gor_actual_vs_target_pct`.

`verify_actual_gor` runs the cylinder-loading bookkeeping (`plan_loading`) in reverse: given the oil and gas volumes actually metered into the transfer cylinder, it recovers the as-loaded GOR and grades its deviation from the target GOR.

$$n_{actual} = \frac{V_{gas,actual} \cdot P_{load}}{Z_{load} \cdot R \cdot T_{load}} \quad V_{std} = n_{actual} \cdot Z_{std} \cdot R \cdot \frac{T_{std}}{P_{std}}$$
$$STO_{actual} = V_{oil,actual} \cdot \frac{\rho_{load}}{\rho_{60F}} \quad GOR = \frac{V_{std}}{STO_{actual}} \cdot \frac{CC_{STB}}{CC_{SCF}}$$
$$dev_{pct} = \frac{GOR - GOR_{target}}{GOR_{target}} \times 100$$

```
check_id = "gor_actual_vs_target_pct"
review_at, fail_at = registry.get(check_id)
severity = grade(dev_pct, review_at, fail_at)
```

Pressure basis note: $P_{load_psia} = psig + 14.73$ (constants.P_STD_PSIA), not $psig + 14.696$ (constants.P_ATM_PSIA), because the Loading_Volumes sheet's gauge-to-absolute formulas add the lab volumetric standard, not the atmosphere/gas-constant standard used elsewhere for psig-to-psia conversions.

Default threshold band: **review at 5.0%, fail at 10.0%.**

8.6 ThresholdRegistry Defaults

Full table, pvt/qc/engine.py, ThresholdRegistry.DEFAULTS:

check_id	review_at	fail_at	Provenance	Implemented?
composition_sum	0.5	2.0	ADRIC house convention	Yes, composition_normalization
mass_balance_pct	2.0	3.0	ADRIC house convention	No check module yet
molar_balance_pct	2.0	3.0	ADRIC house convention	No check module yet
z_deviation_pct	2.0	5.0	ADRIC house convention	No check module yet
density_rsd_pct	0.5	1.0	ADRIC house convention	No check module yet
viscosity_vs_sim_pct	2.0	5.0	ADRIC house convention	No check module yet
mmp_mass_balance_pct	5.0	5.0	ADRIC house convention	No check module yet
gor_actual_vs_target_pct	5.0	10.0	ADRIC house convention	Yes, pvt.experiments.recombination.LC
mw_consistency_pct	5.0	10.0	ADRIC house convention	Yes, mw_consistency
hoffman_r2	0.98	0.95	Engineering judgment, pending Swej calibration (R ² -floor semantics, not a deviation band)	Yes, hoffman_crump

Every threshold not marked “pending Swej calibration” is transcribed from ADRIC house conventions; hoffman_r2 is the sole documented exception, because the source PVT-check sheets show the Hoffman-Crump crossplot visually with no numeric acceptance gate of their own.

8.7 How Pages Surface QC

The pill

ui/common/components.py renders one QCResult as a colored-dot pill:

```
_QC_COLORS = {
    "PASS": TOKENS["qc_green"],    # #38a169
    "REVIEW": TOKENS["qc_amber"],  # #dd9a0a
    "FAIL": TOKENS["qc_red"],      # #e53e3e
}
```

```
def qc_pill(result: QCResult) -> None:
```



```
color = _QC_COLORS[result.severity.value]
# colored dot + check_id (bold) + message (muted, smaller)
```

qc_panel(results) renders a stack of pills, one per QCResult. Both the Flash Separation page (ui/pages/flash_page.py) and the Recombination page (ui/pages/recombination_page.py) call qc_panel under a bold **Composition QC** / **Actual GOR QC** markdown label.

Captions and warnings for skipped checks

The two pages differ in how granularly they surface a skipped check, and both differences trace directly to which InputValidationError is being caught.

Flash page (ui/pages/flash_page.py): the six composition checks (gas/oil mol%/wt% normalization, gas/oil mw_consistency) are run **independently**, each in its own try/except InputValidationError, rather than as one list comprehension. This matters because a mol-only manual-entry composition (no wt% at all) makes mw_consistency raise InputValidationError (it needs both bases); catching that per-check, rather than around the whole batch, means the other five checks (including the Hoffmann-Crump crossplot, which only needs mol%) still run and render. A skipped individual check renders a small caption instead of vanishing silently:

```
except InputValidationError as exc:
    st.caption(f"{label}: skipped - {'; '.join(exc.errors)}")
```

The Hoffmann-Crump section (labelled **Hoffmann-Crump K-value Consistency** in the UI, note the UI spells it with a double n where the module hoffman_crump.py and its check_id/HoffmanResult spell it with one) has its own precheck: if fewer than 2 components overlap between the gas and oil streams, st.warning("Hoffmann-Crump QC skipped: fewer than 2 components present in both streams.") is shown up front. Beyond that precheck, any other InputValidationError from hoffman_crump.check (including the degenerate-fit cases) is caught and shown the same way:

```
except InputValidationError as exc:
    st.warning("Hoffmann-Crump QC skipped: " + "; ".join(exc.errors))
```

When it succeeds, the pill for hoffman.qc renders, followed by an st.scatter_chart of the observed vs. fitted cross-plot points.

Recombination page (ui/pages/recombination_page.py): the two composition-normalization checks (STO mol%, gas mol%) are wrapped in **one** try/except, so a failure skips the whole **Composition QC** block with a single warning:

```
except InputValidationError as exc:
    st.warning("Composition QC skipped: " + "; ".join(exc.errors))
```

mw_consistency is not run on this page at all, not even inside a try/except, because the LiveOil v4.1 importer only reads the Mol% (INPUT) column (block C, section 7.4); the resulting streams never carry a wt% basis, so the check would always raise. The code comment on this page notes it is “skipped entirely rather than caught-and-hidden on every single run.”

The loading plan and actual-GOR verification are similarly guarded, but with except (InputValidationError, ZeroDivisionError), since plan_loading/verify_actual_gor can also divide by a caller-supplied zero (e.g. a target GOR of 0):

```
except (InputValidationError, ZeroDivisionError) as exc:
    st.warning(f"Loading plan unavailable: {exc}")
```

The verify-GOR QCResult, once computed, renders through qc_panel([verify_qc]) under **Actual GOR QC**, and is appended to the same qc_results list that feeds the report download (Chapter 9).

Chapter 9: Reporting

9.1 The ReportRow / ReportTable Model

Module: `pvt/reporting/tables.py`. Two small frozen dataclasses carry every report, regardless of which experiment produced it or which format (Excel, in-app pill list, CLI text) ultimately renders it:

```
@dataclass(frozen=True)
class ReportRow:
    label: str
    value: str
    unit: str = ""

@dataclass(frozen=True)
class ReportTable:
    title: str
    rows: list[ReportRow]
```

A `ReportRow` is a single labelled value, already formatted as a string (the builder functions below choose the decimal precision per field, e.g. `f"{results.v_press_cc:.4f}"`), plus an optional unit string. A `ReportTable` is a titled block of rows, one report section. A report is simply `list[ReportTable]`.

Every `ReportTable` list built by this module ends with a "QC Summary" section, built by the shared helper:

```
def _qc_rows(qc: list[QCResult]) -> list[ReportRow]:
    return [ReportRow(result.check_id, result.severity.value, result.message) for result in qc]
```

One row per `QCResult`: label is the `check_id`, value is the severity string ("PASS"/"REVIEW"/"FAIL"), unit holds the full check message. Every `QCResult` a caller passes in is guaranteed to appear as its own row, nothing is filtered or deduplicated at this layer.

9.2 flash_tables

```
def flash_tables(results: FlashResults, recomb: MassRecombination, qc: list[QCResult]) -> list[ReportTable]
```

Builds three sections from a flash-separation run (`pvt.experiments.flash.calc.calculate`) and its mass-basis recombination (`pvt.experiments.flash.recombine.recombine_mass`):

“Flash Results” (12 rows, from `FlashResults`):

Label	Source field	Format	Unit
Charge Pressure Volume	<code>v_press_cc</code>	<code>.4f</code>	cc
Flashed Oil Mass	<code>m_oil_g</code>	<code>.2f</code>	g
Gas Volume (Measured)	<code>v_gas_meas_cc</code>	<code>.2f</code>	cc
Gas Volume (Standard)	<code>v_gas_std_cc</code>	<code>.2f</code>	cc
Gas Density (Standard)	<code>gas_density_std_g_cc</code>	<code>.6f</code>	g/cc

Label	Source field	Format	Unit
Flashed Gas Mass	m_gas_g	.5f	g
GOR	gor_cc_cc	.4f	cc/cc
GOR	gor_scf_bbl	.2f	scf/bbl
Bo	bo_flash	.4f	vol/vol
Shrinkage	shrinkage	.4f	(none)
Oil Density (60F)	oil_density_60f_g_cc	.4f	g/cc
API Gravity	api	.1f	API

“Whole Sample” (3 rows, from MassRecombination):

Label	Source field	Format	Unit
Gas Mass Fraction	wf_gas * 100.0	.2f	wt%
Oil Mass Fraction	wf_oil * 100.0	.2f	wt%
Whole Sample MW	mw_whole_sample	.2f	g/mol

“QC Summary”: `_qc_rows(qc)`, one row per QCResult passed in.

9.3 recombination_tables

def recombination_tables(split: MolarSplit, plan: LoadingPlan, qc: list[QCResult]) -> list[ReportTable]

Builds three sections from a molar gas/oil split (`pvt.experiments.recombination.molar.molar_split`) and a cylinder loading plan (`pvt.experiments.recombination.loading.plan_loading`):

“Molar Split” (9 rows, from MolarSplit):

Label	Source field	Format	Unit
GOR (Effective)	gor_scf_stb_effective	.2f	scf/STB
GOR	gor_cc_cc	.4f	cc/cc
Gas Moles per cc STO	n_gas_per_cc_sto	.6f	mol/cc
Oil Moles per cc STO	n_oil_per_cc_sto	.6f	mol/cc
Gas Mole Fraction	f_gas * 100.0	.2f	mol%
Oil Mole Fraction	f_oil * 100.0	.2f	mol%
Gas Mass Fraction	w_gas * 100.0	.2f	wt%
Oil Mass Fraction	w_oil * 100.0	.2f	wt%
Wellstream MW	mw_wellstream	.2f	g/mol

“Loading Plan” (10 rows, from LoadingPlan):

Label	Source field	Format	Unit
Oil Charge Volume	v_oil_charge_cc	.2f	cc
STO Equivalent Volume	v_sto_equivalent_cc	.2f	cc
Oil Moles Charged	n_oil_mol	.6f	mol
Gas Moles Required	n_gas_mol	.6f	mol
Gas Volume (Standard)	v_gas_std_cc	.2f	cc
Std cc per cc (Load)	std_cc_per_cc_at_load	.4f	(none)
Gas Charge Volume	v_gas_charge_cc	.2f	cc
Total Charge Volume	total_charge_cc	.2f	cc

Label	Source field	Format	Unit
Fits Cylinder	"Yes" if <code>plan.fits</code> else "No"		(none)
Cylinder Utilization	<code>utilization_pct</code>	.2f	%

“QC Summary”: `_qc_rows(qc)`, one row per `QCResult` passed in.

9.4 Excel Export Styling

Module: `pvt/reporting/excel_export.py`. `write_report(path, tables, *, title, sample)` writes a single-sheet, ADRIC-styled workbook via `openpyxl`. `path` accepts a filesystem path or any writable binary file-like object (e.g. an in-memory `BytesIO`), because `Workbook.save` accepts either transparently.

Colors

```
_NAVY = "00205B"
_WHITE = "FFFFFF"
```

```
_SEVERITY_FILL = {
    "PASS": "38A169",
    "REVIEW": "DD9A0A",
    "FAIL": "E53E3E",
}
```

- **ADRIC navy header:** the title banner cell (row 1) uses `_HEADER_FONT = Font(color=_WHITE, bold=True, size=14)` on `_HEADER_FILL = PatternFill("solid", fgColor=_NAVY)`, i.e. hex 00205B, merged across all three columns.
- **Severity fills:** any data row whose *value* cell (column B) reads exactly "PASS", "REVIEW", or "FAIL" gets that cell filled solid with the matching hex color: green 38A169 for PASS, amber DD9A0A for REVIEW, red E53E3E for FAIL. This is a plain string match in `_write_row`, so it fires for QC Summary rows and any other row whose formatted value happens to equal one of those three strings.

Column layout

Every row is written as three columns: label (A), value (B), unit (C). Section titles and the title banner are single cells merged across all three columns (`_LAST_COLUMN = 3`).

Top to bottom, a written report is:

1. Navy title banner (row 1), merged A:C.
2. Spacer row.
3. “**Sample Information**” section (bold, merged title row), then 9 rows built from the `Sample` passed in: Sample ID, Well, Field, Reservoir, Depth (MD) in ft (formatted .1f, or "N/A" if None), Fluid Type, Cylinder, Client, Project.
4. Spacer row.
5. Each `ReportTable` in the `tables` argument, in order: a bold section-title row, then each of its rows as label/value/unit, then a spacer row.

Column widths are fixed: A = 30, B = 20, C = 50. Column C was widened from an earlier value of 16 specifically because QC Summary rows put the full check message in this column (e.g. "Hoffman-Crump crossplot $R^2=0.9657$ over 3 points (REVIEW)"), and at 16 characters wide Excel’s on-screen display clipped the message (the stored cell value itself was never truncated, only its rendering).

9.5 Downloads in the App

ui/common/components.py's report_download(tables, sample, filename, *, title=None) is what every page's "Download Excel Report" button calls. The workbook is built **entirely in memory**, never touching disk:

```
buffer = BytesIO()
write_report(buffer, tables, title=report_title, sample=sample)
buffer.seek(0)
st.download_button(
    "Download Excel Report",
    data=buffer,
    file_name=_prefixed_filename(filename, sample.sample_id),
    mime="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
)
```

The report title defaults to the base filename, Title-Cased with underscores turned to spaces (_derive_title, e.g. "flash_separation_report.xlsx" becomes "Flash Separation Report"), unless the caller supplies an explicit title.

The download filename is prefixed with the sample ID before it reaches the browser:

```
def _prefixed_filename(filename: str, sample_id: str) -> str:
    prefix = "_".join(sample_id.split()) or "sample"
    return f"{prefix}_{filename}"
```

Whitespace in sample_id is collapsed to underscores. This exists so that reports downloaded for different samples do not share a download name and silently clobber each other in a browser's downloads folder.

9.6 CLI Text Reports

cli.py has two subcommands; only one of them uses the ReportTable/ReportRow model above.

flash subcommand

```
python cli.py flash --workbook path/to/ADRIC_Flash_Separation_Calc_v6.1.xlsx
```

_run_flash imports the workbook via pvt.io.excel_import.flash_v61.read, runs calculate() and recombine_mass(), builds exactly four composition-normalization checks and two mw_consistency checks (gas mol%, gas wt%, oil mol%, oil wt% normalization; gas and oil MW consistency, no Hoffman-Crump crossplot and no GOR verification in this path), calls flash_tables(results, recomb, qc), and prints the result as fixed-width text via _format_report_tables:

```
_COL = 26  # label column width
_W = 52  # total line width (inside borders)

def _rule(char="="):
    return char * (_W + 4)

def _row(label, value):
    return f" {label:<{_COL}}: {value}"

def _section(title):
    pad = (_W - len(title)) // 2
    return f"\n{'-' * (pad + 2)} {title} {'-' * (_W - pad - len(title) + 2)}"
```

_format_report_tables(tables, sample, title) renders: a top = rule and the title, a SAMPLE section (Sample ID, Well, Field, Client, read straight off the Sample), then each ReportTable as an upper-cased, dash-bracketed section

header followed by its rows rendered "{value} {unit}".strip(), then a closing = rule. On import failure, InputValidationError's .errors are printed one per line to stderr (error: <message>) and the command exits 1; an OSError (e.g. file not found) is reported the same way.

recombine subcommand

```
python cli.py recombine --gor 850 --p_sep 815 --t_sep 145 --z_sep 0.855 \  
    --v_live 300 --p_recomb 5014.7 --t_recomb 200 --z_recomb 0.820
```

This is the original single-purpose CLI (multi-stage separator recombination, Carlsen & Whitson SF/FF flow), predating the ReportTable model. It builds its fixed-width report by hand, appending directly to a lines: list[str], using the same _rule/_row/_section helpers, rather than constructing ReportTable/ReportRow objects. Its sections are: SETUP, RECOMBINATION CONDITIONS, one STAGE block per separator stage, CHARGE VOLUMES, VERIFICATION (back-calculated GOR match, with a checkmark or a Δ deviation X% flag if the round-trip error exceeds 0.1%), and, when both --api and --sg_gas are supplied, a BUBBLE POINT ESTIMATE section from Standing's (1947) correlation. The report closes with a fixed footer line: Standard conditions: 14.73 psia (lab basis) / 60 °F.

10. Deviations Ledger and Roadmap

10.1 The deviations-ledger discipline

The engine’s authority over calculation logic is unconditional: where a source ADRIC workbook is provably wrong, or takes a stance the engine does not, the engine implements what the underlying petroleum-engineering reference says is correct, and the difference is recorded — not silently ported, not silently corrected. The record lives in one file, `docs/excel-deviations.md`, and every entry is added before the deviating behavior’s test lands. Any golden-parity test failure that is *not* backed by a ledger entry is treated as a bug in the port, not a deliberate deviation; this is the mechanism that keeps “the engine fixed it” from becoming an unaudited claim.

Lifecycle

Every entry starts life as proposed. It stays proposed until it has been walked through point-by-point with the PVT domain owner (SweJ), at which point it resolves to one of:

- **approved** — the engine’s behavior is confirmed correct and the workbook’s differing behavior is confirmed a defect.
- **parity-kept** — the workbook’s behavior is confirmed the intended lab convention, and the engine is expected to match it (the ledger entry documents the convention rather than a bug).

As of this writing every entry in the ledger is still proposed; none has been walked through the phase-wrap review yet. D-018 additionally carries `NEEDS SWEJ RULING` — it is not just unreviewed, it is a live open question (see §10.3).

What a ledger row contains

Each row in the `docs/excel-deviations.md` table has five fields:

Field	Contents
ID	D-NNN, assigned in the order the deviation was found
Workbook / cell	The source file and the specific sheet/cell(s) or formula the behavior was read from — the proof that the claim is verifiable, not asserted
Excel behavior	What the workbook actually computes, in enough detail to reproduce it
Engine behavior	What pvt/ computes instead, with a citation to the published correlation/reference where one exists, and the numeric size of the divergence where it has been measured
Status	proposed, approved, or parity-kept

10.2 Current ledger: D-001 through D-018

Fifteen entries exist today. The IDs are not contiguous: D-012, D-013, and D-014 do not appear in `docs/excel-deviations.md` — there is a gap in the numbering, not fifteen consecutive rows. Every row below is still proposed.

ID	Summary
D-001	Two component-library MW variants circulated (LiveOil v4.1 vs Flash v6.1); the engine canonizes one table (Flash v6.1 values) with C36+ MW as the only per-study override.
D-002	Workbook sheets never call their own validators, risking silent divide-by-zero; engine calc entry points raise InputValidationError instead (Phase 2 pattern, recombination retrofit tracked separately).
D-003	Piper-McCain-Corredor CO2 coefficient is transposed in the reference workbook ($\alpha_2 = -0.09034$); engine uses the published -0.90348 (SPE 26668, 1993), shifting Tpc $\sim 2.5\%$ at 20% CO2.
D-004	Reference workbook computes C7+ MW as an unweighted sum/average; engine mole-fraction-weights it, $\text{Sigma}(y_i \cdot \text{MW}_i) / \text{Sigma}(y_i)$, per SPE 26668.
D-005	Reference workbook's Dranchuk-Abou-Kassem Newton iteration uses an incorrect derivative of the residual; engine uses the mathematically correct derivative (converged Z roots are identical; only the iteration path differs).
D-006	CVD workbook's Additional_QC Hall-Yarborough variant uses Tr instead of the reciprocal t, omits t from the A-term, and reports the reduced density itself as "Z"; engine implements canonical Hall & Yarborough (1973), matching the separate, correct Gas_Gradient VBA kernel.
D-007	Standing bubble-point sheet leaves the correlation exponent a as a raw user-entered cell rather than computing it; engine computes a from T/API internally, with a bubble_point_with_exponent() parity hook that reproduces the sheet's raw-a form exactly.
D-008	Vasquez-Beggs sheet computes the exponent term by multiplying by (T+460) instead of dividing, overflowing to #NUM!; engine divides, per the published Pb-form (Vasquez & Beggs 1980), cross-checked against the original Rs-form inversion ($\sim 0.5\%$ agreement).
D-009	Glaso sheet multiplies Pb* by a stray 14.5 and never applies the final $10^{\log_{10}(\text{Pb})}$ step, and rounds the correlation constants; engine applies the published constants (1.7669/1.7447) and the correct final exponentiation, anchored to the workbook's own cached Pb* cell to prove the stray factor is understood, not guessed.
D-010	Viscosity workbook hardcodes the lbm/ft3-to-g/cc conversion as the rounded 0.0014935; engine derives the coefficient exactly from canonical constants (0.0014926...), $\sim 0.06\%$ lower.
D-011	Reference VBA hardcodes the gas constant as 10.73; engine uses the canonical $R_{\text{PSIA_FT3_LBMOL_R}} = 10.7316$, per the task brief; all other Jossi-Stiel-Thodos coefficients are transcribed from the VBA exactly.
D-015	The old pre-Phase-0 app used $P_{\text{std}} = 14.696$ psia and $\text{CC_PER_STB}} = 158987.1$; engine uses the canonical lab basis $P_{\text{STD_PSIA}} = 14.73 / \text{CC_PER_STB}} = 158987.29$ per the design spec, shifting recombination outputs $\sim 0.2\%$ versus the old app.
D-016	Gas_Gradient VBA's EstimatePseudoCriticals takes density in g/cc and converts internally; engine's erbar.c7_plus_criticals takes SG directly (the VBA's post-conversion value) — callers with raw density must convert first.
D-017	Flash v6.1 computes a P_base (barometric + back-pressure) that is never used downstream; V_gas_std actually uses the separately measured absolute pressure. Engine takes the measured absolute pressure input only and does not compute the unused quantity.

LiveOil v4.1's GOR-basis toggle divides the **Stock Tank** GOR by shrinkage when convention divides the **Separator** GOR; masked in the workbook today because its shrinkage factor is 1.0. Engine implements the conventional direction. **Open — needs a ruling; see §10.3.**

10.3 Open rulings

Four items remain genuinely undecided — not unreviewed line items, but questions where the workbook, the engine, and lab convention do not all point the same way. These are carried over verbatim from docs/workbook-defect-review.md §3:

1. **D-018 — GOR-basis direction.** LiveOil v4.1's Recombination!B8 divides the Stock Tank GOR by shrinkage; the engine currently implements the conventional direction (Separator GOR / SF -> STO basis). The workbook's own goldens do not distinguish the two because its shrinkage factor is 1.0, so this has not yet been forced to a decision by test data. Needs a ruling before D-018 can move past proposed.
2. **hoffman_r2 bands.** The Hoffman-Crump QC check currently grades $R^2 \geq 0.98$ as PASS and $R^2 \geq 0.95$ as REVIEW. These are engineering-judgment defaults, not yet confirmed against house practice.
3. **Z = 1 conventions.** Both the Flash v6.1 workbook (Volumetrics_Master!B27) and the MSS workbook (MSS_Input!024:027, gas Z collected but never used) assume $Z = 1$ in standard-volume gas conversions — a ~0.2-0.3% systematic bias that is standard lab-sheet practice but currently undocumented as a deliberate convention. Whether to keep it as-is or wire in an actual Z correlation (sheet-side or platform-side) is open.
4. **DV pressure basis and MMP recovery basis.** The DV workbook (3_ADRIC_DV_Calc_v6.2.xlsx) enters Psat in psig with no +14.7 conversion anywhere downstream, while the engine uses absolute pressure consistently — the house convention for stage pressures needs to be fixed explicitly. Separately, the MMP recovery-basis defect (dead volume double-counted or omitted between the QC tool and the raw-logger file) needs a defined house basis; the engine currently exposes this as an explicit flag rather than picking silently.

Two related housekeeping items from the same review are worth noting here without being open engineering questions: flipping the fifteen ledger entries from proposed to approved/parity-kept is itself pending the walk-through described in §10.1, and the stashed `stash@{0}` (a July 15 cosmetic ASCII-diagram tweak to the pre-Phase-2 `ui/recombination.py`) is still sitting untouched, per the design spec's git-workflow section, until the UI rebuild absorbs or formally obsoletes it.

10.4 Phase roadmap

The phased scope below is defined in docs/superpowers/specs/2026-08-09-pvt-lab-platform-design.md §2. Phases 0-2 are built (see the earlier chapters of this manual); Phases 3-5 are planned. Each phase's paragraph below draws on what the full formula-level dissection of the ADRIC workbook set — docs/workbook-defect-review.md — already tells us about the modules that phase will need to implement.

Phase 3 — CCE, DV, MSS, Density HPHT, Viscosity HPHT

Phase 3 covers Constant Composition Expansion (with Y-function and compressibility QC), Differential Vaporization (with the Amyx/Carlson flash-basis adjustment), Multi-Stage Separator tests, and the two HPHT physical-property tests (density and viscosity). The dissected workbooks already surface most of what this phase has to get right on day one. The CCE workbook (2_CCE_Calculation_Sheet_v5...xlsx) has a mean-compressibility sign flip that propagates a negative value into its own report tab, "PVTsim comparison" columns that are vacuous self-references rather than real comparison data, and a spurious compressibility spike at the bubble-point row from a central-difference stencil that straddles the two-phase transition; its Y-function is QC-only by design across every version from v1 to v5, never used to auto-pick Psat, which the platform should keep as deliberate rather than "fix." Two QC checks that existed in the v1/v2 CCE sheets — compressibility monotonicity and $\rho \cdot V$ constancy — were dropped in v5's layout compaction and need to be resurrected in the platform's QC set. The DV workbook has a bubble-point Rs formula that wrongly folds in stage-2's delta-Rs, an empty-link Fluid Type field, Psat entered in psig with no conversion to the absolute-pressure basis

the rest of the sheet needs, dead override cells for STO density/MW, only eight gas composition slots (data beyond that silently drops from the mass balance), and a back-pressure term that is never added to the measured gasometer pressure. The MSS workbook repeats the Fluid Type link bug, collects a gas Z column it never uses, has a K-value trend block that divides by an always-empty reservoir-composition column so its charts are permanently blank, and has no cross-test check comparing its own measured cumulative GOR against the flash R_{s_fb} value it should be validated against — exactly the kind of check the Phase 5 QC Center is meant to formalize. The two HPHT sheets share a machine-written `_xludf.STDEV.5` artifact that silently evaluates to blank, meaning Std Dev, %RSD, and the QC pass/fail verdict have never actually computed in either sheet; the density sheet’s working pressure and temperature inputs are wired to nothing (water density enters only via two manually typed values, a candidate for computing $\rho_w(P,T)$ from IAPWS instead) and its average includes out-of-range runs; the viscosity sheet’s table lookups require an exact pressure match and a hardcoded row index for the stock-tank row, both of which the engine should interpolate/derive rather than replicate.

Phase 4 — CVD, MMP

Phase 4 covers Constant Volume Depletion (Whitson-style material balance) and MMP slim-tube analysis, including a real two-line-intersection solver Excel never had. The CVD workbook (`CVD_Calc_WaterPump_v3_WhitsonStyle_AdditionalQC.xlsx`, which ships empty) has two defects serious enough to flag as critical before any lab use of the sheet itself: cells labeled “R” and “scf_per_lbmol” are missing a sheet qualifier and actually resolve to that sheet’s own stage-pressure cells, poisoning every downstream mole-count calculation, and a stage table referenced at the wrong row offset drops four stages from every chart and breaks the report’s dewpoint/temperature figures. Its `Additional_QC` block has the same broken Hall-Yarborough variant already handled by D-006, plus a Hoffmann K-value estimate that overflows to `#NUM!` from an ungaurded 10^x , and several physical constants stored as text rather than numbers. Because the CVD workbook ships empty, Phase 4’s golden fixtures for it will necessarily be synthetic — a published dataset (e.g. Whitson & Brulé) frozen into `tests/fixtures/`, not a lab-cached value — until real lab data is available. The MMP tooling’s most consequential finding is structural rather than a formula bug: MMP itself is never computed anywhere in the existing tools — the engineer eyeballs the intersection of two plotted lines and types in the answer — so Phase 4’s slim-tube module needs an actual solver, not a port. Beyond that, the recovery-percent calculation double-counts or omits dead volume depending on which of two inconsistent formulas is used (the platform should expose this as an explicit, named basis rather than silently picking one), empty runs produce a garbage “100% error / REVIEW” result instead of being excluded, the gas and oil series in the raw-logger template use forward and backward differences respectively (a one-sample GOR lead the engine must not reproduce), and the reporting lookup uses a stair-step `VLOOKUP(..., TRUE)` where real interpolation is needed.

Phase 5 — Cross-test QC Center, consolidated reporting

Phase 5 implements the full PVT-check QC catalog as a first-class cross-test QC Center, plus consolidated study-level reporting. The case for this phase is made by nearly every workbook reviewed: CCE’s dropped monotonicity and ρV -constancy checks, MSS’s missing GOR cross-check against the flash-derived R_{s_fb} , and the DV/MSS Fluid Type link bugs are all instances of the same underlying gap — checks that exist in principle (or existed in an earlier sheet version) but are not wired to run automatically today, and nothing in the current tooling compares a result computed in one test against the same physical quantity computed in another. The reference-tool review (`docs/workbook-defect-review.md §2`) adds the specific check catalog this phase should implement: Hoffmann ($\log_{10}(KP)$ vs F linearity), Buckley ($\ln K$ vs T_c^2 linearity), Y-function regression fit and per-point error, K-value trend consistency, material- and molar-balance closure, GOR back-calculation from a material-balance diagram slope, and a compressibility-sign screen ($Co/C_g \geq 0$). The design spec’s Study/CrossRef data model (already in place since Phase 0) is what makes this possible without manual retyping: results computed upstream (CCE’s P_{sat} , for instance) are consumed downstream with provenance, so the QC Center can flag staleness the moment an upstream value changes rather than relying on an engineer to notice a stale hand-typed cell, which is exactly the failure mode several of the defects above describe.